Digital Pendulum System

REFERENCE MANUAL

33-005-2M5

Matlab 5 Version

Feedback

Feedback Instruments Ltd, Park Road, Crowborough, E. Sussex, TN6 2QR, UK.
Telephone: +44 (0) 1892 653322, Fax: +44 (0) 1892 663719.
email: feedback@fdbk.co.uk website: http://www.fbk.com

Manual: 33-005-2M5 Ed02 062000 *Printed in England by FI Ltd, Crowborough*
Feedback Part No. 1160–330052M5

Notes

THE HEALTH AND SAFETY AT WORK ACT 1974

We are required under the Health and Safety at Work Act 1974, to make available to users of this equipment certain information regarding its safe use.

The equipment, when used in normal or prescribed applications within the parameters set for its mechanical and electrical performance, should not cause any danger or hazard to health or safety if normal engineering practices are observed and they are used in accordance with the instructions supplied.

If, in specific cases, circumstances exist in which a potential hazard may be brought about by careless or improper use, these will be pointed out and the necessary precautions emphasised.

While we provide the fullest possible user information relating to the proper use of this equipment, if there is any doubt whatsoever about any aspect, the user should contact the Product Safety Officer at Feedback Instruments Limited, Crowborough.

This equipment should not be used by inexperienced users unless they are under supervision.

We are required by European Directives to indicate on our equipment panels certain areas and warnings that require attention by the user. These have been indicated in the specified way by yellow labels with black printing, the meaning of any labels that may be fixed to the instrument are shown below:

CAUTION - RISK OF DANGER

Refer to accompanying documents

CAUTION - RISK OF ELECTRIC SHOCK

CAUTION - ELECTROSTATIC SENSITIVE DEVICE

PRODUCT IMPROVEMENTS

We maintain a policy of continuous product improvement by incorporating the latest developments and components into our equipment, even up to the time of dispatch.

All major changes are incorporated into up-dated editions of our manuals and this manual was believed to be correct at the time of printing. However, some product changes which do not affect the instructional capability of the equipment, may not be included until it is necessary to incorporate other significant changes.

COMPONENT REPLACEMENT

Where components are of a ‘Safety Critical’ nature, i.e. all components involved with the supply or carrying of voltages at supply potential or higher, these must be replaced with components of equal international safety approval in order to maintain full equipment safety.

In order to maintain compliance with international directives, all replacement components should be identical to those originally supplied.

Any component may be ordered direct from Feedback or its agents by quoting the following information:

1. Equipment type
2. Component value
3. Component reference
4. Equipment serial number

Components can often be replaced by alternatives available locally, however we cannot therefore guarantee continued performance either to published specification or compliance with international standards.

DECLARATION CONCERNING ELECTROMAGNETIC COMPATIBILITY

Should this equipment be used outside the classroom, laboratory study area or similar such place for which it is designed and sold then Feedback Instruments Ltd hereby states that conformity with the protection requirements of the European Community Electromagnetic Compatibility Directive (89/336/EEC) may be invalidated and could lead to prosecution.

This equipment, when operated in accordance with the supplied documentation, does not cause electromagnetic disturbance outside its immediate electromagnetic environment.

COPYRIGHT NOTICE

© Feedback Instruments Limited

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without the prior permission of Feedback Instruments Limited.

ACKNOWLEDGEMENTS

Feedback Instruments Ltd acknowledge all trademarks.

IBM, IBM - PC are registered trademarks of International Business Machines.

MICROSOFT, WINDOWS 95, WINDOWS 3.1 are registered trademarks of Microsoft Corporation.

MATLAB is a registered trademark of Mathworks Inc.

TABLE OF CONTENTS

1 Introduction 1-1

2 Description of the Toolbox Functions 2-1

2.1 GetAlgNo 2-2

2.2 GetBaseAddress 2-3

2.3 GetCartPosFilt 2-5

2.4 GetCartSpeedFilt 2-6

2.5 GetDesPosition 2-7

2.6 GetDivider 2-9

2.7 GetHistory 2-10

2.8 GetNoOfSamples 2-15

2.9 GetP 2-16

2.10 GetPendPosFilt 2-17

2.11 GetPendSpeedFilt 2-18

2.12 GetPW 2-19

2.13 GetSampleTime 2-20

2.14 LoadLibrary 2-21

2.15 ResetEncoder 2-23

2.16 ResetTime 2-24

2.17 SetAlgNo 2-26

2.18 SetBaseAddress 2-28

2.19 SetCartPosFilt 2-29

2.20 SetCartSpeedFilt 2-30

2.21 SetDesPosition 2-31

2.22 SetDivider 2-33

2.23 SetP **2-35**

2.24 SetPendPosFilt **2-38**

2.25 SetPendSpeedFilt **2-43**

2.26 SetPW **2-44**

2.27 SetSampleTime **2-48**

2.28 StartAcq **2-49**

2.29 StopPractical **2-50**

2.30 UnloadLibrary **2-52**

3 Information Flow between Simulink Models and the Real-Time Kernel **3-1**

3.1 Simulink models for MATLAB version 4 **3-2**
3.1.1 Guide for Simulink models and S-functions (examples) 3-2
3.1.2 S-function - example 1 3-5
3.1.3 S-function - example 2 3-7

3.2 Simulink models for MATLAB version 5 **3-14**
3.2.1 Guide for Simulink models and S-functions (examples) 3-14
3.2.2 S-function - example 1 3-17
3.2.3 S-function - example 2 3-22

4 Guide to the Graphical User Interface (Examples) **4-1**

4.1 Types of Controls **4-2**

4.2 How to create and use controls (examples) **4-3**

5 Quick Reference Table **5-1**

1 Introduction

The software for the Digital Pendulum system consists of:

- **A Real-Time Kernel (RTK),**
- **Communication functions,**
- **Pendulum Toolboxes,**
- **External Interface,**
- **Windows NT kernel-mode device driver (for Windows NT only).**

The Real-Time Kernel supervises a set of real-time tasks creating a feedback control structure (Fig. 1.1). The RTK provides a mechanism for real-time measurements and control of the digital -pendulum system in the MS-WINDOWS environment. It has the form of a dynamic linked library (DLL) and is written in closed form. No modifications are necessary. RTK contains all functions which are required for control and data collection in real-time.

There are **two different RTKs** for the different versions of MATLAB and different operating systems, which must be selected during software installation time.

1) RTK for MATLAB ver. 5.x and Windows 95,

2) RTK for MATLAB ver. 5.x and Windows NT operating system.

Control algorithms are embedded in the real-time kernel and their selection and tuning of parameters is done by means of the **communication interface** from the MATLAB environment. The communication interface is also used for configuration of real-time kernel parameters (sampling period, excitation source etc.).

Specialised **communication functions** are responsible for communication between RTK and MATLAB environment These functions are described in this manual.

The **Pendulum Toolbox** consists of a collection of M-functions (among them are S-functions as well), Simulink models and C-code DLL-files that extend the MATLAB environment in order to solve pendulum modelling/design/control problems. A cyclic memory buffer is created in order to enable process data flow between the real-time kernel and toolbox functions (MATLAB environment). The Pendulum Toolbox, using MATLAB matrix functions and Simulink utilities, provides the user with functions specific for the real-time control of the digital pendulum system.

The toolbox is designed as an **open system** (as compared to the RTK which is a closed system). This approach makes the basic functions of the toolbox available to the user and enables the user to create a tailored made system, add to it new control algorithms and subsequently customise it further to satisfy the requirements.

However, there are also demonstration M-files for typical pendulum control problems available in the toolbox. And these are described in the Digital Pendulum System GETTING STARTED Manual (33-005-1M5)

The External Interface for the digital pendulum system offers a way for ***the more experienced user*** to extend the capabilities of the software. Although the RTK and MATLAB interface create a complete, self contained environment for real-time experiments, its applicability is limited to a fixed number of embedded controllers. The interface provides a way for the user to design and add new controllers to the RTK and implement them in real-time.

The **kernel-mode device driver** which gives the RTK access to the I/O address space is required in the Windows NT environment.

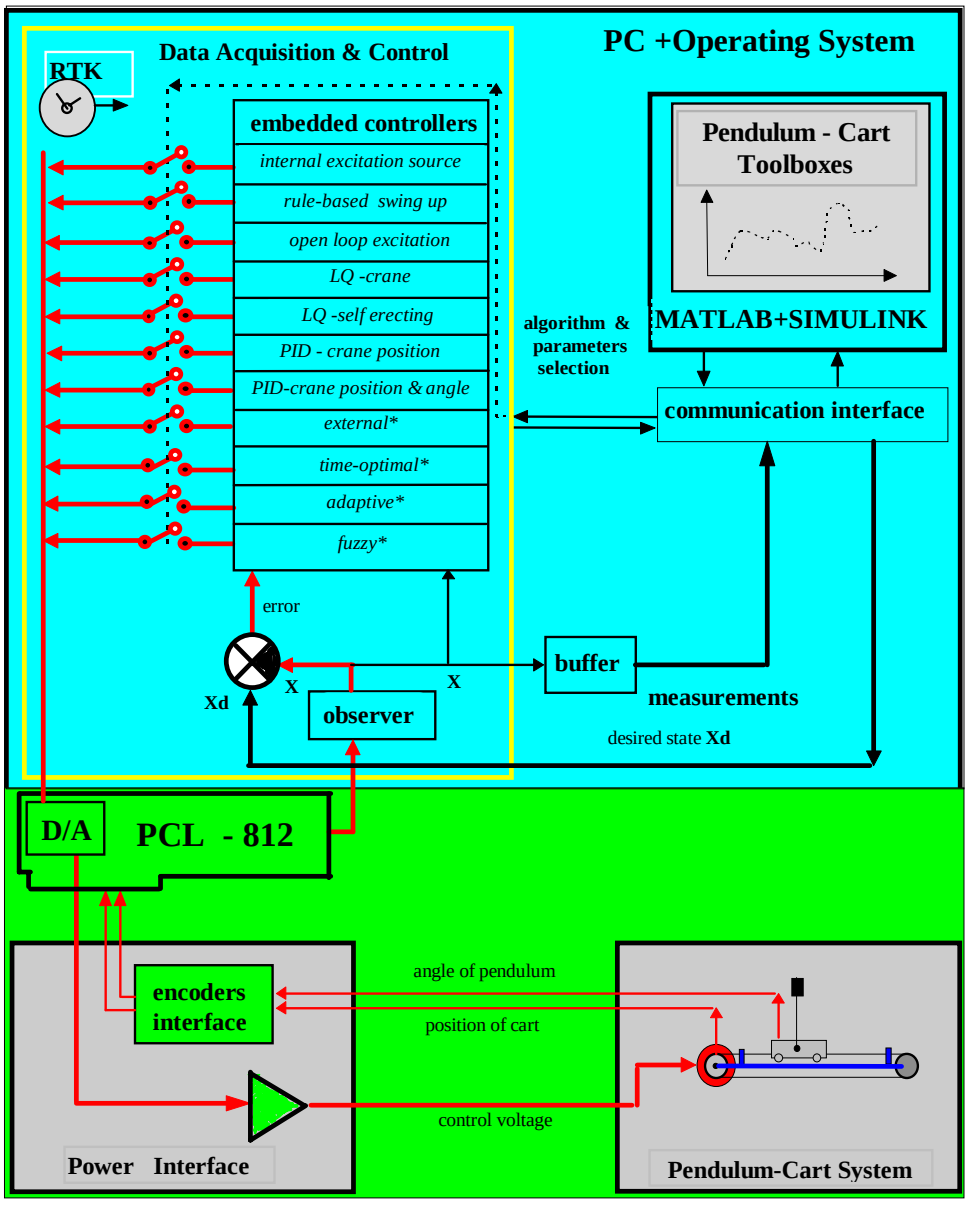
The following pages contain a description of the Communication Toolbox functions. There are 21 Communication Toolbox functions which are listed in alphabetic order and which are divided into the four following categories according to their specific assigned roles.

- hardware: *GetBaseAddress, SetBaseAddress, ResetEncoder*
- data acquisition: *GetNoOfSamples, ResetTime, GetSampleTime, SetSampleTime, StartAcq, GetHistory*
- software: *LoadLibrary, UnloadLibrary*
- control: all ten remaining functions.

The following format is used in this manual:

Purpose	Provides a short description of the function
Logo	Refers to the block diagram of the control system in Fig. 1.1.
Synopsis	Shows the format of the function call.
Description	Describes what the function does and any restrictions that apply.
Arguments	Describes the arguments of the function.
See	Refers to other related functions.
Examples	Provides examples of how the function can be used
Notes	Optionally remarks
MATLAB 5	Informs about limited applicability of the description. The description refers only to MATLAB version 5
Windows NT	Describes those features which are only applicable to Windows NT

All DLLs used by the inverted pendulum system must be located in the MATLAB root directory. Execute the **matlabroot** ***command in the MATLAB Command Window to find the root directory of the MATLAB program. Remember that during experiments the MATLAB the current directory must be the same as the MATLAB root directory. Use the*** **pwd** ***MATLAB command to check the name of the current directory and if necessary, use the*** **cd** ***command in the MATLAB Command Window to change.***



* option

Fig. 1.1 Digital control of pendulum (hardware and software configuration)

2 Description of the Toolbox Functions

All functions responsible for communication between the Real-Time Kernel (RTK) and MATLAB environment are in the following form:

ReturnValue = pd_call(FunctionName, [Argument])

where:

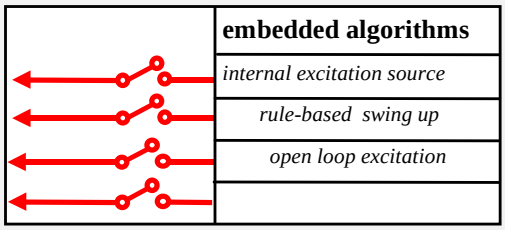
ReturnValue - value returned by the function,

pd_call - name of the DLL library responsible for the communication,

FunctionName - name of the desired operation, string format,

Argument - argument passed to the *pd_call* function (optional).

2.1 GetAlgNo



Purpose: Get number of the currently active control algorithm.

Synopsis: *AlgNo = pd_call('GetAlgNo')*

Description: The function returns the number of the algorithm currently active in RTK.

See: SetAlgNo

Example: The following function displays the description of the currently active control algorithm.

```
function dspdsr
   AlgNo = pd_call( 'GetAlgNo' );
   if AlgNo == 0
     disp( 'Control set to zero' );
   elseif AlgNo == 1
     disp( 'Open-loop control' );
   elseif AlgNo == 2
     disp( 'Rule-based controller' );
   elseif AlgNo == 3
     disp( 'LQ crane mode controller' );
   elseif AlgNo == 4
     disp( 'Fuzzy controller' );
   elseif AlgNo == 5
     disp( 'PID crane mode controller' );
   elseif AlgNo == 99
     disp( 'External controller' );
   else
     disp( '!!! Unknown controller !!!' );
   end
```

2.2 GetBaseAddress

PCL-812

Purpose: Get base address of PCL-812 board.

Synopsis: *BaseAddr = pd_call('GetBaseAddress')*

Description: The function is called to obtain the base address of PCL-812 interface board. This mode is useful to test communication between RTK and MATLAB without connection to external hardware.

See: *SetBaseAddress*

Note: If the *GetBaseAddress* function returns a base address is equal to zero the RTK generates dummy data. See Example 2 for details.

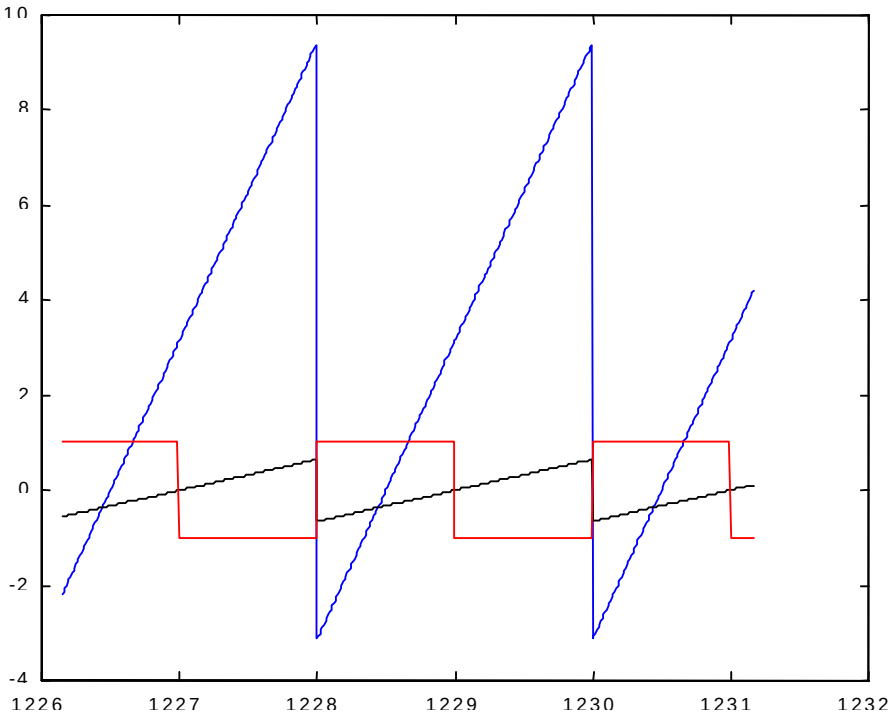
Example 1:

```
BaseAddr = pd_call( 'GetBaseAddress' )
BaseAddr =
        544
```

Example 2*:* The base address equal to zero can be used to test the software. This operating mode is useful to test real-time software or to test MATLAB m-scripts and Simulink models. It does not require any interface hardware.

The following commands display the example of dummy data.

```
ret = pd_call( 'SetBaseAddress', 0 );
ret = pd_call( 'StartAcq' );  pause( 5 )
hist = pd_call( 'GetHistory' );
plot( hist( 1, : ), hist( [2 4 6], : ) )
```



2.3 GetCartPosFilt

Purpose: Get the parameters of the cart position digital filter.

Synopsis*:* *fi = pd_call('GetCartPosFilt')*

Description: The function gets the parameters of the digital filter currently active in RTK. The filter associated with the *GetCartPosFilt* function operates on the data describing the position of the cart on the rail.

The filter structure is described by the difference equation:

$$y(t) = a_0 x(t) + \sum_{i=1}^{8} a_i x(t - iT_0) + \sum_{i=1}^{8} b_i y(t - iT_0)\,,$$

where:

$y(t)$ - filter output,

$x(t)$ - filter input,

T_0 - sampling period,

$a_0,\ldots,a_8,b_1,\ldots,b_8$ - parameters of the filter.

All filters implemented in RTK have a_0 parameter set to one and all others parameters set to zero as default. It gives the transfer function for the filter equal to 1 (the output signal is equal to the input signal).

The return variable *fi* is a 2-by-9 matrix in the form:

$$\begin{bmatrix} a_0 & a_1 & a_2 & a_3 & a_4 & a_5 & a_6 & a_7 & a_8 \\ 0 & b_1 & b_2 & b_3 & b_4 & b_5 & b_6 & b_7 & b_8 \end{bmatrix}.$$

See: *GetCartSpeedFilt, SetCartPosFilt*

2.4 GetCartSpeedFilt

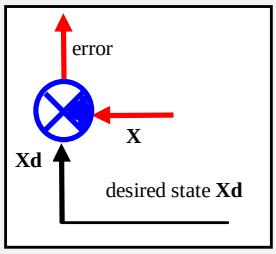
Purpose: Get the parameters of the cart velocity digital filter.

Synopsis*:* *fi = pd_call('GetCartSpeedFilt')*

Description: The function gets the parameters of the digital filter currently active in RTK. The filter associated with the *GetCartSpeedFilt* function operates on the data describing the velocity of the cart on the rail. See description of the *GetCartPosFilt* for filter details.

See: *GetCartPosFilt, SetCartSpeedFilt*

2.5 GetDesPosition



Purpose: Get desired position of the cart.

Synopsis*:* *DesCartP = pd_call('GetDesPosition')*

Description: The function returns the desired position of the cart. The cart position is calculated in relation to the position set by *ResetEncoders* function (usually it is centre of the rail). The position is scaled in meters.

See: *SetDesPosition, GetHistory*

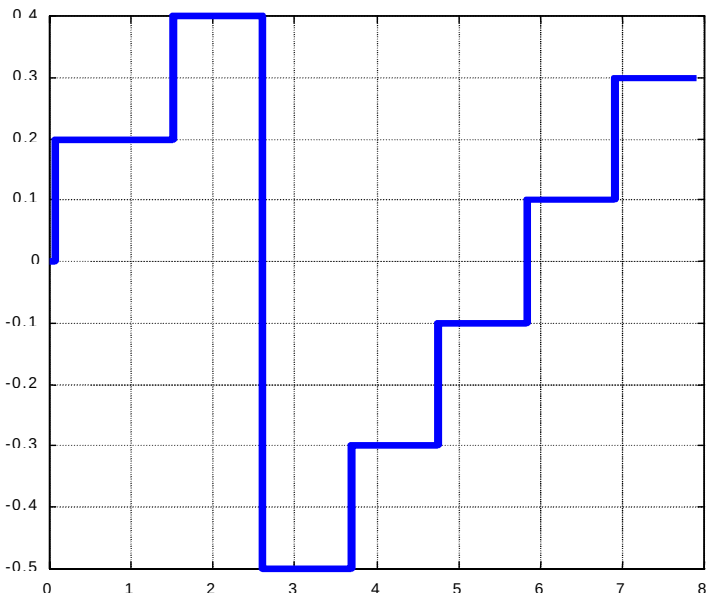
Example 1: *DesPos = pd_call('GetDesPosition')*

DesPos =

0.15

Example 2: The following commands change desired position of the cart every second. The history of changes is displayed in the diagram below

```
ret = pd_call( 'SetDesPosition', 0.0 );
ret = pd_call( 'StartAcq' );
for i = 1 : 7
        despos = pd_call( 'GetDesPosition' );
        despos = despos + 0.2;
        if despos > 0.5
        despos = -0.5;
  end
  ret = pd_call( 'SetDesPosition', despos );
  pause( 1 );
  end
  hist = pd_call( 'GetHistory' );
  stairs( hist( 1, : ) - hist( 1, 1 ), ...
        hist( 7, : ) )
  grid
```



2.6 GetDivider

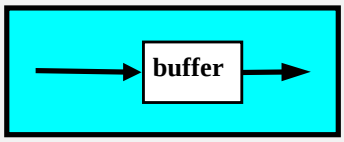
Purpose: Get divider of the auxiliary clock.

Synopsis: *Div = pd_call('GetDivider')*

Description: The function returns the divider of the basic RTK clock

See: *SetDivider, GetSampleTime, SetSampleTime*

2.7 GetHistory



Purpose: Get content of the internal RTK buffer.

Synopsis*:* *Hist = pd_call('GetHistory')*

Description: The function returns *Hist* matrix containing the history of an experiment and sets the buffer to zero. The structure of matrix *Hist* is given in Table 1:

Table 1.

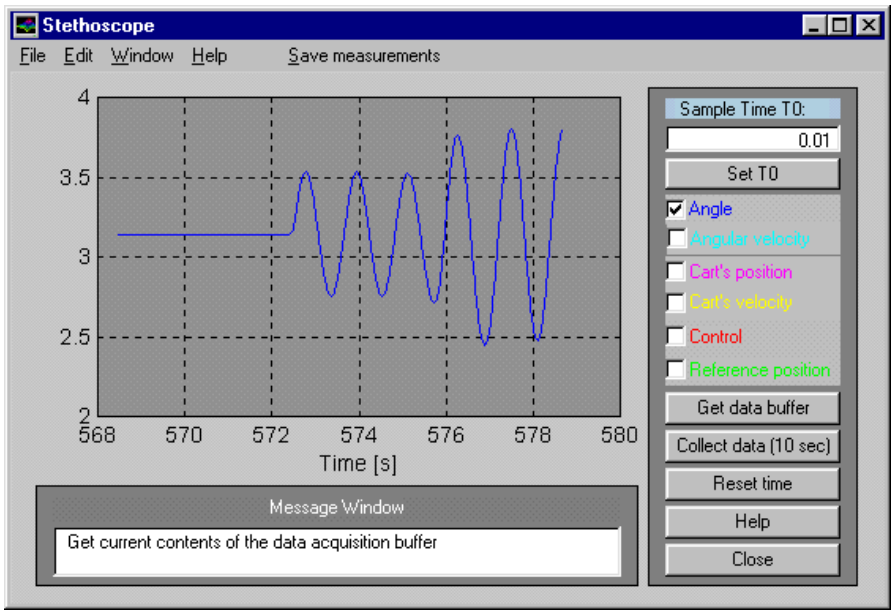
Row of the matrix	Description	Units
Hist(1,:)	regularly spaced time	seconds
Hist(2,:)	angle of the pendulum	radians
Hist(3,:)	angular velocity of the pendulum	radians/seconds
Hist(4,:)	position of the cart	meters
Hist(5,:)	velocity of the cart	meters/seconds
Hist(6,:)	control	relative units in the range ±1
Hist(7,:)	desired position of the cart	meters

Note: The maximum number of samples available in the buffer is 1000.

See*:* *GetNoOfSamples, StartAcq*

MATLAB 5

Note: The GUI tool is available which displays data collected by the RTK. Execute the *pd_stet* command in the MATLAB command window to call this tool (see figure below).



Example: Read 250 data points and display the data.

```
pd_call( 'StartAcq' );                    % clear data buffer
while( pd_call( 'GetNoOfSamples' ) < 250 ) ;% wait for data
end;
Hist = pd_call( 'GetHistory' );           % get data and
                                          % clear the buffer
```

To plot the angle of the pendulum execute the following command:

```
plot( Hist( 1, : ), Hist( 2, : ) ); grid
```

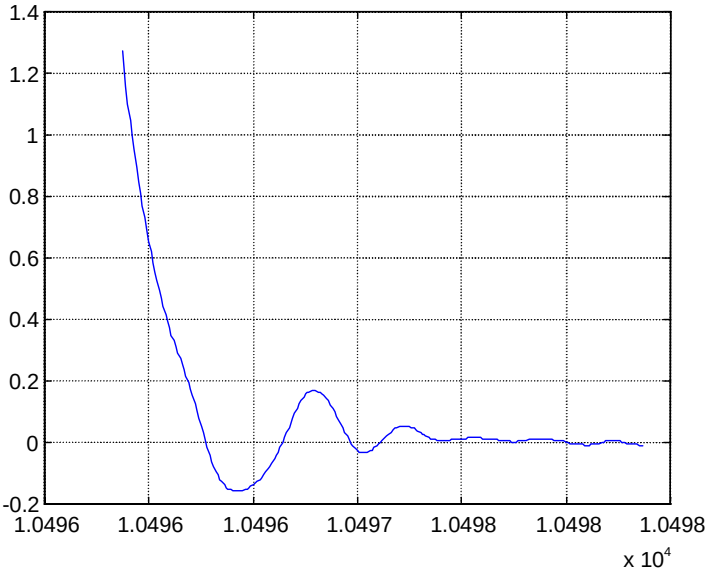


Fig. 2.1 Angle vs. time.

Fig. 2.1 shows the angle vs. time diagram. Notice that time starts from 10496 seconds. This means that the experiment was started after 10496 seconds from the moment when *pd_dll* library has been loaded to the memory. If you want to plot the time from zero, modify the command : *plot(Hist(1, :)-Hist(1, 1),Hist(2, :)); grid*

Fig. 2.2 shows the plot.

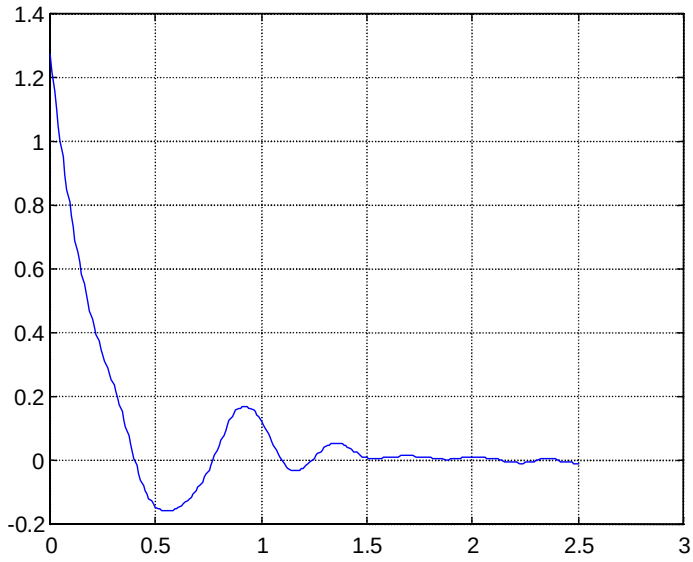


Fig. 2.2 Angle vs. time (plot started from zero).

The following commands plots angle and angular velocity vs. time (see Fig: 2.3):

plot(Hist(1, :), Hist(2, :), Hist(1, :), Hist(3, :)); grid

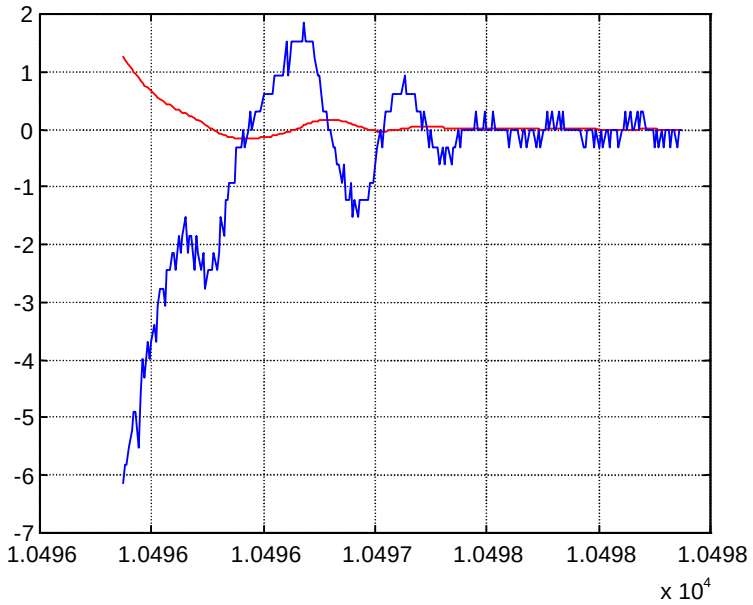


Fig. 2.3 Angle and angular velocity vs. time.

To show position of the cart and cart velocity execute the following command (Fig. 2.4):

plot(Hist(1, :), Hist(4, :), Hist(1, :), Hist(5, :)); grid

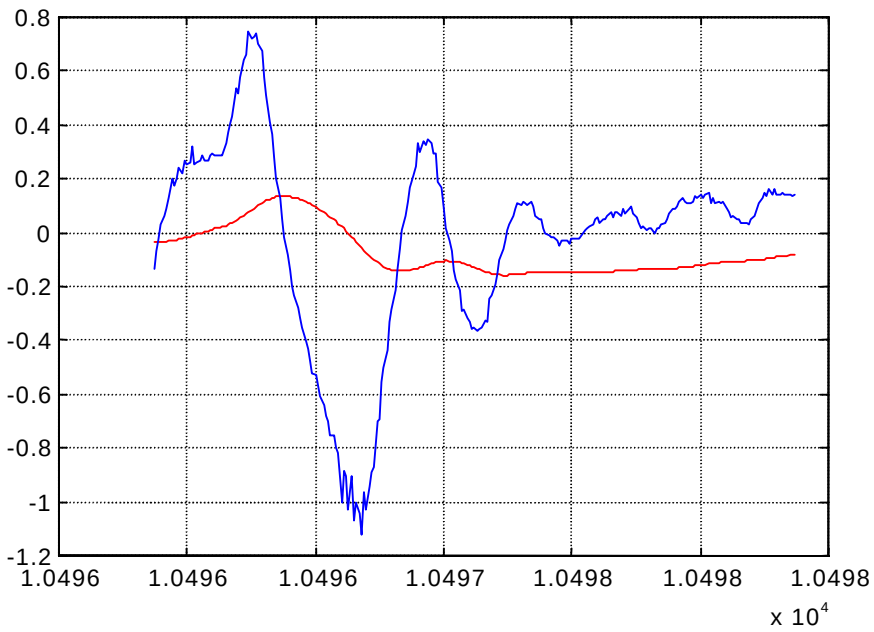


Fig. 2.4. Position and velocity of the cart vs. time.

The following command plots control vs time(Fig. 2.5):

plot(Hist(1, :), Hist(6, :))

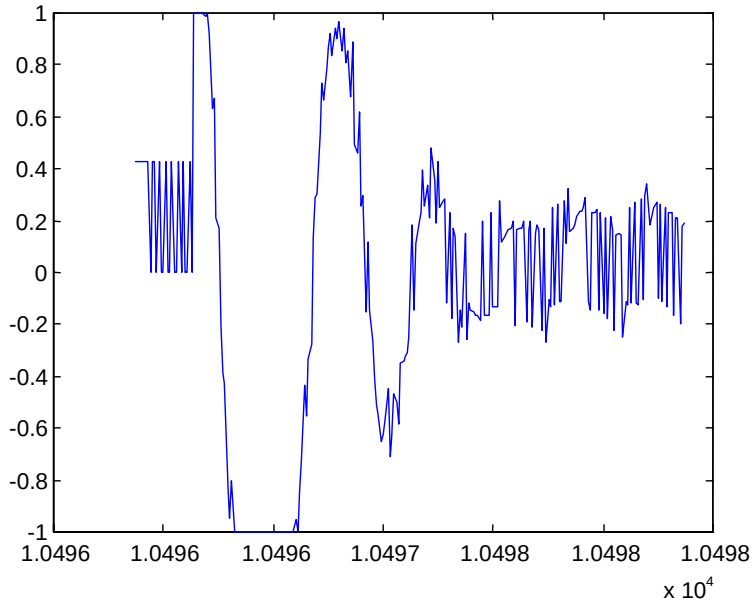
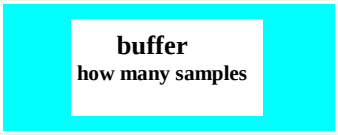


Fig. 2.5. Control vs. time.

2.8 GetNoOfSamples



Purpose: Get number of samples available in the buffer.

Synopsis: *Hist = pd_call('GetNoOfSamples')*

Description: The function returns the number of samples currently available in the buffer. The maximum number of samples available in the buffer is equal to 1000.

See: *GetHistory*

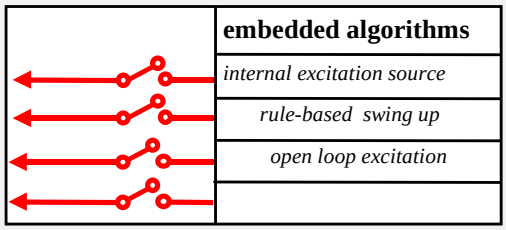
Example 1: Wait until the number of data in the buffer is greater than 120.

```
pd_call( 'StartAcq' );          % clear buffer
                                % buffer's length is set to zero
while( pd_call( 'GetNoOfSamples' ) < 120 ) ;
end;
```

Example 2: The maximum data buffer length is 1000 samples. The following sequence will never terminate:

```
while( pd_call( 'GetNoOfSamples' ) < 1200 ) ;
end;
```

2.9 GetP



Purpose: Get parameters of the control algorithm.

Synopsis: *Par = pd_call('GetP')*

Description: The function returns the vector containing the parameters of the control algorithm currently active in the RTK. The vector contains 20 elements. The set of parameters is common for all algorithms, but only specified values are used by a currently active controller. See the *SetAlgNo* function for details.

See: GetPW, SetP

Example: Increase the second parameter by 120%

```
Par = pd_call( 'GetP' );
Par( 2 ) = 1.2 * Par( 2 );
pd_call( 'SetP', Par );
```

2.10 GetPendPosFilt

Purpose: Get the parameters of the pole position digital filter.

Synopsis: *fi = pd_call('GetPendPosFilt')*

Description: The function gets the parameters of the digital filter currently active in RTK. The filter associated with the *GetPendPosFilt* function operates on the data describing the angle of the pole.

See description of the *GetCartPosFilt* for filter details.

See: *GetCartSpeedFilt, SetCartPosFilt*

2.11 GetPendSpeedFilt

Purpose: Get the parameters of the pole velocity digital filter.

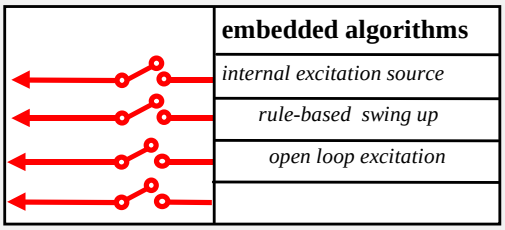
Synopsis: *fi = pd_call('GetPendSpeedFilt')*

Description: The function gets the parameters of the digital filter currently active in RTK. The filter associated with the *GetPendSpeedFilt* function operates on the data describing the velocity of the pole.

See description of the *GetCartPosFilt* for filter details.

See: *GetPendPosFilt, SetPendSpeedFilt*

2.12 GetPW



Purpose: Get parameters of internal excitation source.

Synopsis: *Par = pd_call('GetPW')*

Description: The function returns the vector containing the parameters of internal excitation generator currently active in RTK. The vector contains 20 elements. The set of parameters is common for all shapes of excitation waves, but only some values are used by currently active excitation source.

See: *SetPW*

Example: Set and read parameters of internal excitation source.

```
Par = zeros( 1, 20 );
Par( 1 : 5 ) = [ 1 -1.2 4.66 0 7 ];
pd_call( 'SetPW', Par);
Par = pd_call( 'GetPW' )
Par =
1 -1.2 4.66 0 7 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

2.13 GetSampleTime

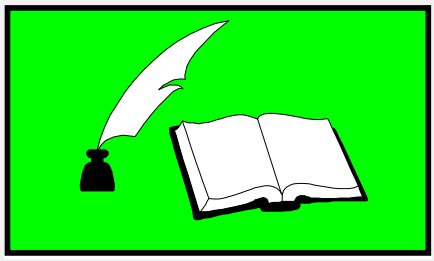
Purpose: Get basic sampling time.

Synopsis: *SmplT = pd_call('GetSampleTime')*

Description: The function returns the period of the basic RTK clock. The period is given in seconds.

See: *SetSampleTime*

2.14 LoadLibrary



Purpose: Load RTK to memory.

Synopsis: *Count = pd_call('LoadLibrary')*

Description: The function loads the *pd_dll.dll* library module. If the *pd_dll.dll* library was loaded previously *LoadLibrary* function only increments (increases by one) the reference counter of the module. The library can not be loaded to the memory again. If the module has not been loaded previously, the function loads it from the *pd_dll.dll file* and sets the reference counter to 1. The *LoadLibrary* increments the reference counter each time MATLAB calls this function. If you want to finish using the module, you should use the *UnloadLibrary* function to decrement (decrease by one) the reference counter. The *pd_dll.dll* library is removed automatically from the memory when the module reference counter reaches zero.

First call to any *pd_call* function loads the *pd_dll.dll* module into memory automatically.

The function returns the module reference counter.

MATLAB 5

In the RTK version for MATLAB ver. 5 the *pd_dll.dll* file does not exist. All functions are included in the *pd_call.dll* module. This module does not contain the reference counter. The first call of the *pd_call* function loads the module into memory. The call to the *UnloadLibrary* function removes the module from memory.

Note: The following messages can appear during execution of the LoadLibrary function:

Can not open PD_PAR.INI file.

The base address is set to zero - the *pd_par.ini* file is not present in the root MATLAB directory. The *pd_par.ini* file should contain the following line which defines base address of the PCL-812PG card:

BaseAddress= 544

If the *pd_par.ini* file is missing the base address of the card is set to zero. It causes the RTK to generate dummy data (see the *GetBaseAddress* function),

Can not find BaseAddress parameter in the *PD_PAR.INI* file - the *pd_par.ini* file is corrupted. See message above for details,

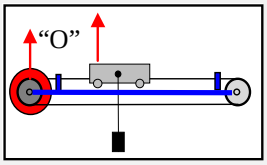
Windows NT

Couldn't access RTKIO device, or Can not start IO access –

These two messages can appear in the Windows NT operating system only. They are caused by absence of the *RTKIO* kernel mode device driver running in the operating system. See the description of installation procedure to solve this problem.

See: *UnloadLibrary*

2.15 ResetEncoder



Purpose: Set incremental encoders to initial position.

Synopsis: *pd_call('ResetEncoder')*

Description: The function sets the initial position of the cart and pendulum. The function must be called when the cart remains motionless in the middle of the rail and the pole hangs in the down position. The *ResetEncoder* function should be executed immediately after loading RTK to memory.

See: *GetDesPosition, LoadLibrary*

2.16 ResetTime

Purpose: Set the experiment time to zero.

Synopsis*:* *pd_call('ResetTime')*

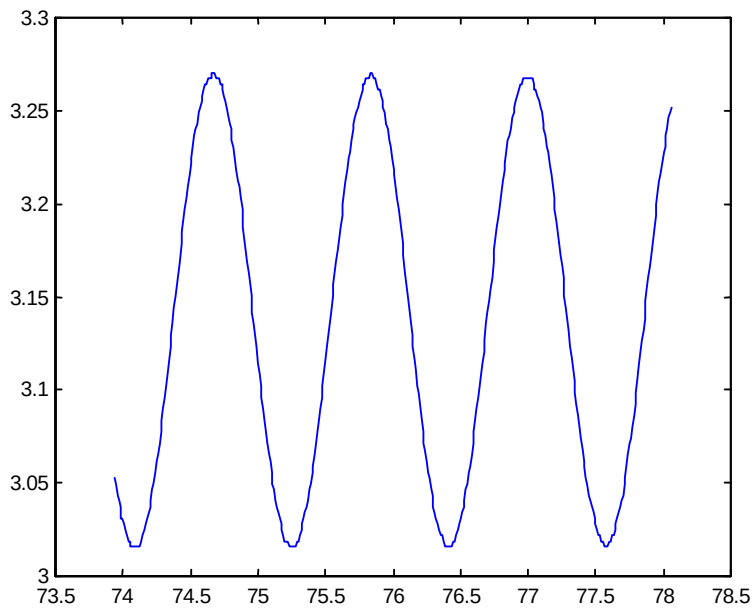
Description: The function sets the experiment time to zero. Time is calculated from the first call of the *pd_call* function. After call of the *ResetTime* function time returned by *GetHistory* function is set to zero.

See: *GetHistory*

Example: Read measurement data and plot angle of the pole:

hist = pd_call('GetHistory');

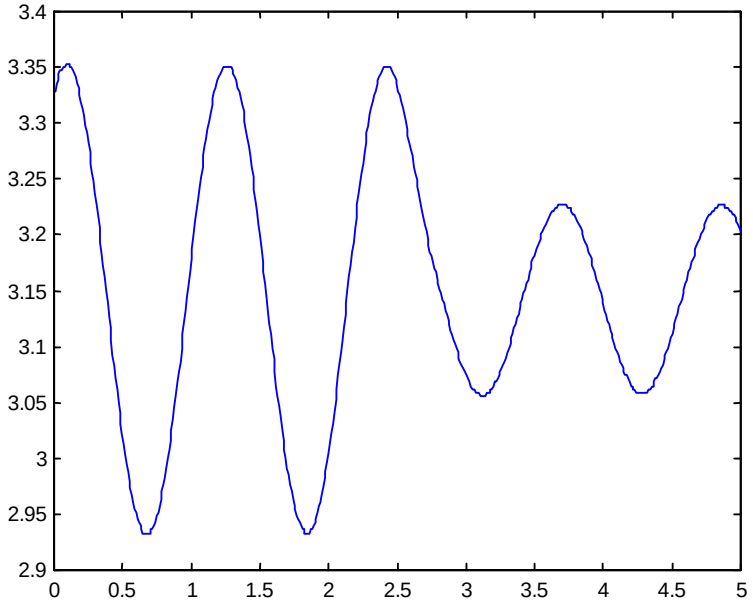
plot(hist(1, :), hist(2, :))



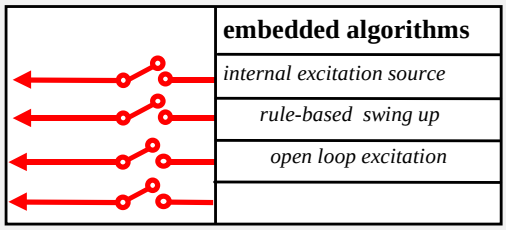
You can notice that time on the x-axis starts from a value equal to 73.5. If you want to start experiment's time from zero execute the following commands:

pd_call('ResetTime'); pause(5); hist = pd_call('GetHistory');

plot(hist(1, :) , h(2, :))



2.17 SetAlgNo



Purpose: Select the control algorithm and start the experiment.

Synopsis: *pd_call('SetAlgNo', AlgNo)*

Description: The function is called to choose the control algorithm in the RTK. The list of available algorithms is given in Table 2. The function starts the experiment. The algorithm must be supplied with appropriate parameters, or default parameters are used. The *SetAlgNo* function must be preceded by a call of the *SetP* or *SetPW* functions. Use the *SetPW* function before *pd_call('SetAlgNo', 1)*, and *SetP* in other cases.

Table 2.

Algorithm no	Description
0	Stop experiment - set control to zero
1	Open loop excitation
2	Rule-based - swing up and LQ upright position stabilisation
3	LQ - down position stabilisation (crane mode)
4*	Fuzzy controller
5	PID - down position stabilisation (crane mode)
6*	Time-optimal controller
7*	Adaptive-controller
99*	External controller

* - extended version of the toolbox

See: *GetAlgNo, GetP, GetPW, SetP, SetPW*

Example 1: Activate the internal excitation source and generate a triangle function. Start an open-loop experiment.

```
pd_call( 'SetAlgNo', 0 );                  % stop experiment
pd_call( 'SetPW', [ 2 5 1 -0.7 0.5] );     % set triangle excitation
pd_call( 'SetAlgNo', 1 );                  % start experiment
pd_call( 'GetAlgNo' )
ans =
                    1
```

Example 2: Set PID controller parameters and start a close-loop experiment.

```
pd_call( 'SetAlgNo', 0 );                  % stop experiment
pd_call( 'SetP', [ 0.1  -3.0  -0.03  0.0  0.5  0.3  0.5 ] );
                                           % friction compensation = 0.1
                                           % Kp = -3,0,  Ki = -0.03,  Kd = 0.0
                                           % rail working range - ±0.5m
                                           % Max.= ±0.3
                                           % integrator limits ±0.5
pd_call( 'SetAlgNo', 5 );                  % begin PID experiment
```

2.18 SetBaseAddress

PCL-812

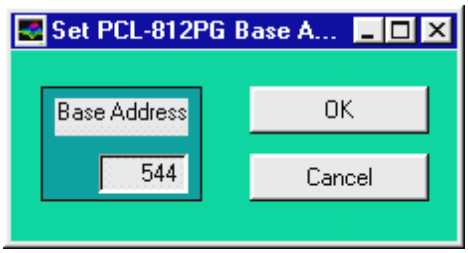
Purpose: Set base address of PCL-812 board.

Synopsis: *pd_call('SetBaseAddress', Address)*

Description: The function is called to set a new base address of PCL-812 board. Notice, that the base address is fixed in the configuration file *pd_par.ini* and is read each time RTK is loaded to memory. Set the address of PCL-812 board to 0. In this case RTK generates dummy date. (See the installation and commissioning manual 33-005-0.)

See: *GetBaseAddress*

Note 1: There is a GUI tool available which can be used to set base address of the PCL-812 board. The tool is called by the *pd_setad* command. The view of the tool is presented below.



Note 2. The base address is loaded from the *pd_par.ini* file during loading the RTK library to the memory. Change the appropriate line in this file to change base address permanently. (The *GetBaseAddress* command sets the new value of the base address *only* in the current session)

Example: Set the address of the PCL-812 board to 544 (220 in hexadecimal code).

```
pd_call( 'SetBaseAddress', 544 );
pd_call( 'GetBaseAddress' )
ans =
    544
```

The following command causes the generation of "dummy" data by the RTK:

```
pd_call( 'SetBaseAddress', 0 );
```

2.19 SetCartPosFilt

Purpose: Set the parameters of the cart position digital filter.

Synopsis: *fi = pd_call('SetCartPosFilt')*

Description: The function sets the parameters of the digital filter currently active in RTK. The filter associated with the *SetCartPosFilt* function operates on the data describing the position of the cart on the rail.

See: description of the *GetCartPosFilt* for filter details.

See: *SetCartSpeedFilt, GetCartPosFilt*

2.20 SetCartSpeedFilt

Purpose: Set the parameters of the cart velocity digital filter.

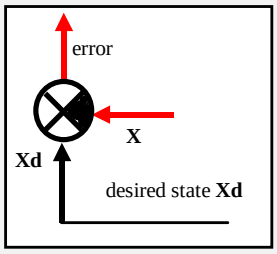
Synopsis*:* *fi = pd_call('SetCartSpeedFilt')*

Description: The function sets the parameters of the digital filter currently active in RTK. The filter associated with the *SetCartSpeedFilt* function operates on the data describing the velocity of the cart on the rail.

See description of the *GetCartPosFilt* for filter details.

See*:* *SetCartPosFilt, GetCartSpeedFilt*

2.21 SetDesPosition



Purpose: Set desired position of the cart.

Synopsis: *pd_call('SetDesPosition', DesPos)*

Description: The function sets the desired position of the cart. The desired position must be in the range of *±0.5* (working range of the rail).

See: *GetDesPosition*

Example: Set LQ controller parameters to stabilise the position and read data from the buffer.

```
pd_call( 'SetAlgNo', 0 );          % stop experiment
pd_call( 'SetP', [ 0.1  -2.0  -1.0  0.0  0.0  0.35  0.5 ] );
                                   % friction compensation - 0.1
                                   % K1 = 3,0,  K2 = 5.0,  K3 = 1.0,  K4 = 1.0
                                   % rail working range - ±0.5m
                                   % Max. = ±0.35m
pd_call( 'SetDesPosition', 0.0 );% set position of the cart
                                   % in the middle of the rail
pd_call( 'SetAlgNo', 3 );          % start LQ controller
pause( 5 )                         % wait 5 seconds
pd_call( 'SetDesPosition', 0.2 );% set position of the cart to 0.2
pause( 5 )
h = pd_call( 'GetHistory' );
```

The following commands plot: angle of the pendulum and angular velocity, position and desired position of the cart, velocity of the cart and control value (Fig.2.6):

```
subplot( 221 );                                   % left upper corner figure
plot( h( 1, : ) - h( 1, 1 ), h( 2, : ), h( 1, : ) - h( 1, 1 ), h( 3, : ) )
subplot( 222 ); );                                % right upper corner figure
plot( h( 1, : ) - h( 1, 1 ), h( 4, : ), h( 1, : ) - h( 1, 1 ), h( 7, : ) )
subplot( 223 ); );                                % left lower corner figure
plot( h( 1, : ) - h( 1, 1 ), h( 5, : ) )
subplot( 224 ); );                                % right lower corner figure
plot( h( 1, : ) - h( 1, 1 ), h( 6, : ) )
```

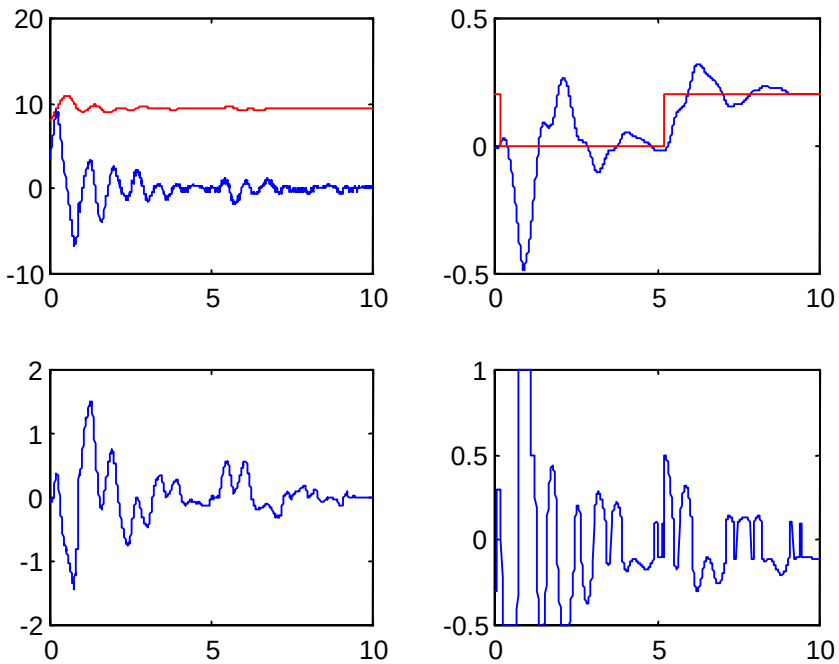


Fig. 2.6. History of the experiment vs. time.

2.22 SetDivider

Purpose: Set clock divider.

Synopsis*:* *pd_call('SetDivider', Div)*

Description: The function sets the auxiliary clock of RTK. This clock can be used by control algorithms. If the *Div* value is greater then 1 the measurements are performed *Div* times faster then the control value is calculated. It creates the possibility to measure more frequently then control signal is calculated.

See*:* *GetDivider*

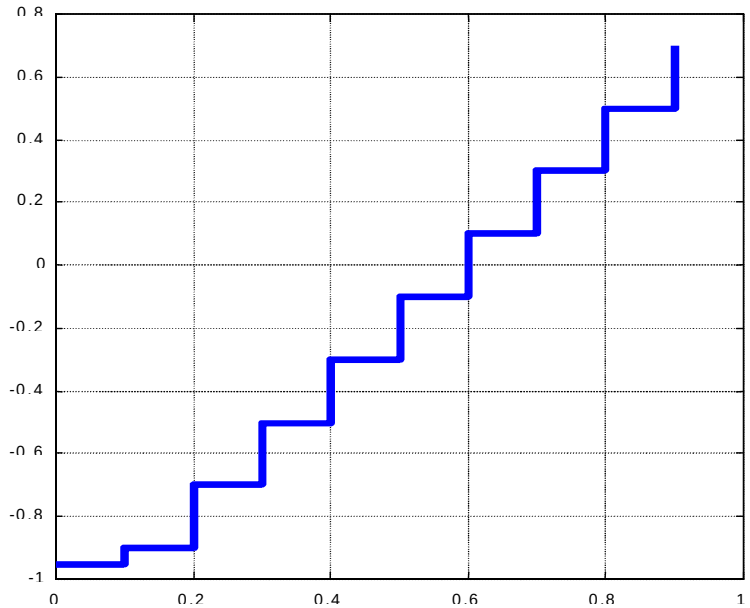
Example 1: Set the control period 3 times longer than the sampling period.

pd_call('SetDivider', 3);

Example 2: The following commands:

```
% set max value of control
pd_call( 'SetP', [ 1 zeros( 1, 19 ) ] );
% select open-loop control
pd_call( 'SetAlgNo',1 );
pd_call( 'SetDivider', 1 );
% set triangle open-loop excitation
pd_call( 'SetPW', [2 1 2 -1 1 zeros( 1, 15 ) ] );
pd_call( 'SetSampleTime', 0.1 );
h = pd_call( 'GetHistory' );
stairs( h( 1, : )-h( 1, 1 ),h( 6, : ) )
```

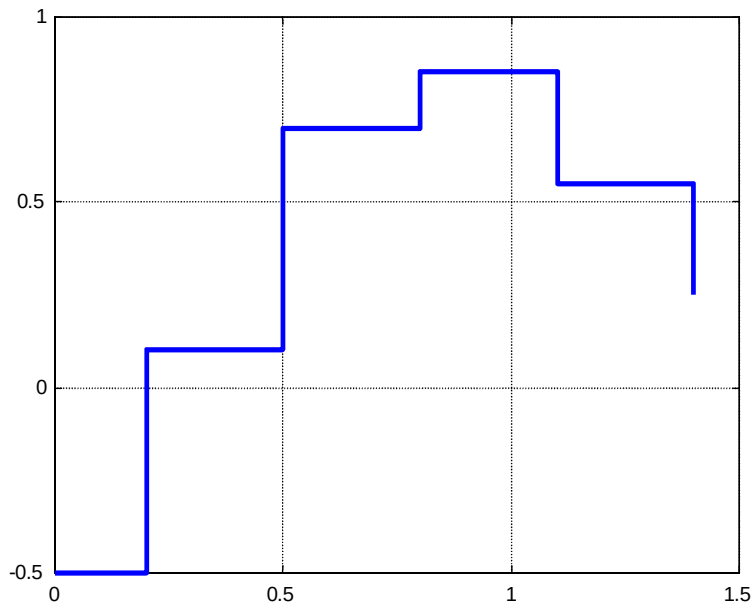
create the following diagram:



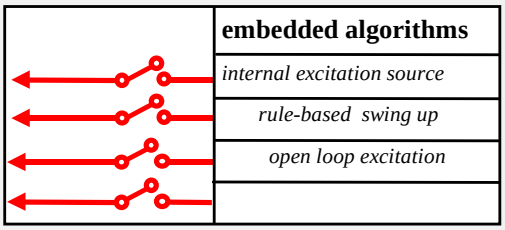
After the command

pd_call('SetDivider', 3);

the picture changes (see figure below). Notice that the control value changes three times slower.



2.23 SetP



Purpose: Set parameters of the controller.

Synopsis*:* *pd_call('SetP', Par)*

Description: The function is called to set parameters of the controller. Next, the number of the algorithm should be selected (*SetAlgNo*). The selected algorithm will be activated with the pre-set parameters. The function *SetP* can be used for all control algorithms. The controller parameters have the following meaning:

for algorithm number 0:

the parameters are not required,

for algorithm number 1:

Par(1) - working range of the rail. The cart moves only within this range. See function *SetPW* for detailed description,

for algorithm number 2:

Par(1)	- static friction compensation
Par(2 : 5)	- K1, K2, K3 and K4 coefficients
Par(6)	- working range of the rail
Par(7)	- maximum, absolute value of the control
Par(8)	- W coefficient
Par(9)	- R value
Par(10)	- stabilisation zone

for algorithm number 3:

Par(1)	- static friction compensation
Par(2 : 5)	- K1, K2, K3 and K4 coefficients
Par(6)	- working range of the rail
Par(7)	- maximum, absolute value of the control

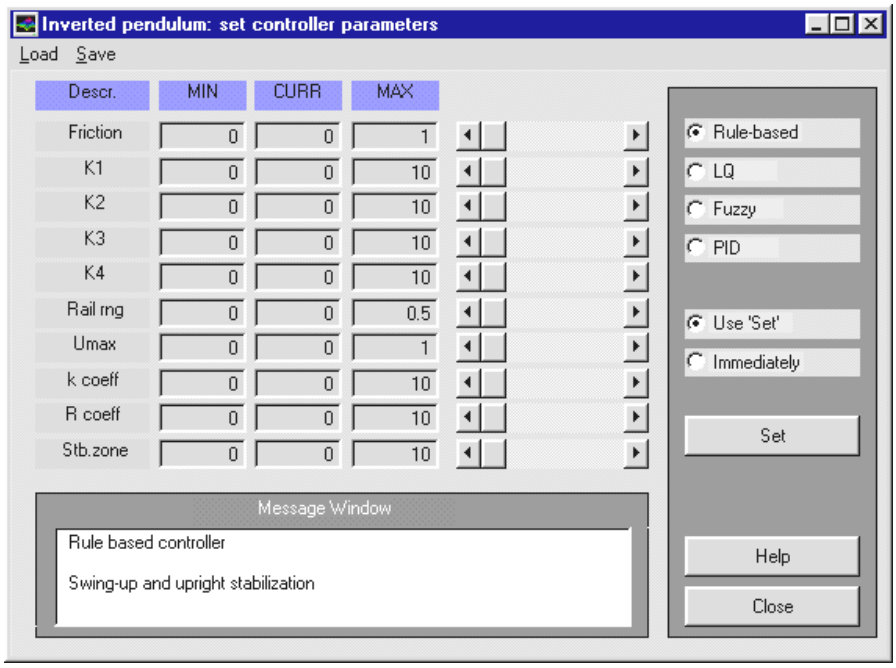
for algorithm number 5:

Par(1)	- static friction compensation
Par(2 : 4)	- Kp, Ki and Kd coefficients
Par(5)	- working range of the rail
Par(6)	- maximum, absolute value of the control
Par(7)	- saturation of the integrator

See: *GetP, SetPW, GetPW*

MATLAB 5

Note: There is a GUI tool which can be used to tune the parameters of the controllers. Execute the *pd_cpar* command to call this tool (see figure below).



Example: Start PI controller.

```
pd_call( 'SetAlgNo', 0 );          % stop excitation
pd_call( 'SetP', [ 0.1  -3.0  -0.03  0.0  0.5 0.3  0.5 ] );
pd_call( 'SetAlgNo', 5 );
```

2.24 SetPendPosFilt

Purpose: Set the parameters of the pole position digital filter.

Synopsis*:* *fi = pd_call('SetPendPosFilt')*

Description: The function sets the parameters of the digital filter currently active in RTK. The filter associated with the *SetPendPosFilt* function operates on the data describing the angle of the pole.

See description of the *GetCartPosFilt* for filter details.

See: *SetCartSpeedFilt, GetPendPosFilt*

Example*:* The default value of transfer function for all built-in filters is equal to 1.

The following command

fi = pd_call('GetPendPosFilt');

reads filter parameters and sets the *fi* variable to

1 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0

In the filter equation:

$$y(t) = a_0 x(t) + \sum_{i=1}^{8} a_i x(t - iT_0) + \sum_{i=1}^{8} b_i y(t - iT_0)\,,$$

where:

$y(t)$ - filter output,

$x(t)$ - filter input,

T_0 - sampling period,

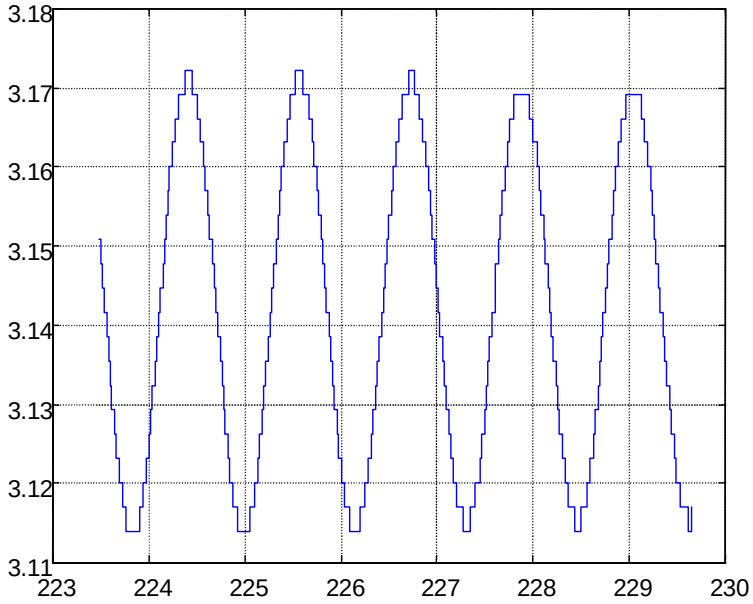
$a_0,\ldots,a_8,b_1,\ldots,b_8$ - parameters of the filter.

the *fi* variable is equivalent to $a_0 = 1$ and other parameters of the filter equal to zero.

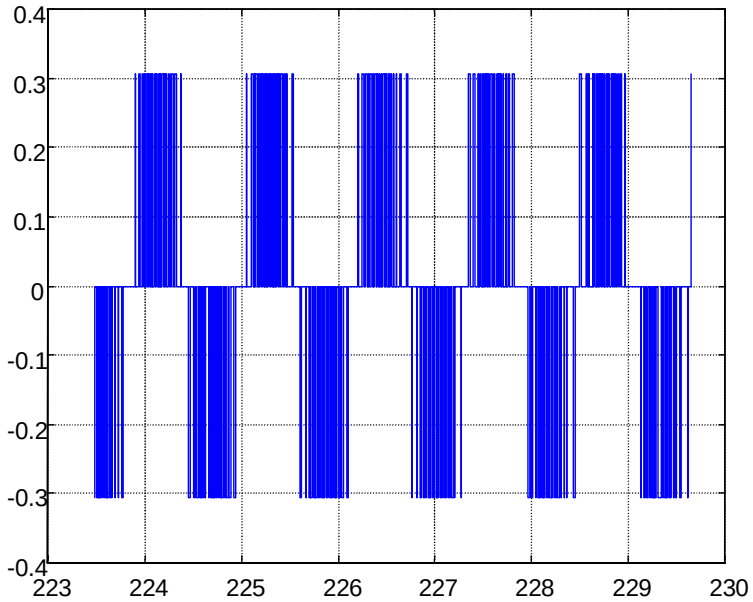
When the pole moves slowly near the downright equilibrium point the following commands read and plot position and velocity of the pole:

h = pd_call('GetHistory');

stairs(h(1, :), h(2, :)); grid



stairs(h(1, :), h(3, :)); grid



The velocity calculated by the RTK is based on the angle data according to the following formula:

$$velocity(t) = \frac{angle(t) - angle(t - T_0)}{T_0},$$

where:

T_0 is the sampling period.

Such a simple formula can give a "strange" looking velocity data, as presented above. To improve quality of the velocity observation the filtering can be used. The following statements define the third-order filter:

fi = pd_call('GetPendPosFilt');

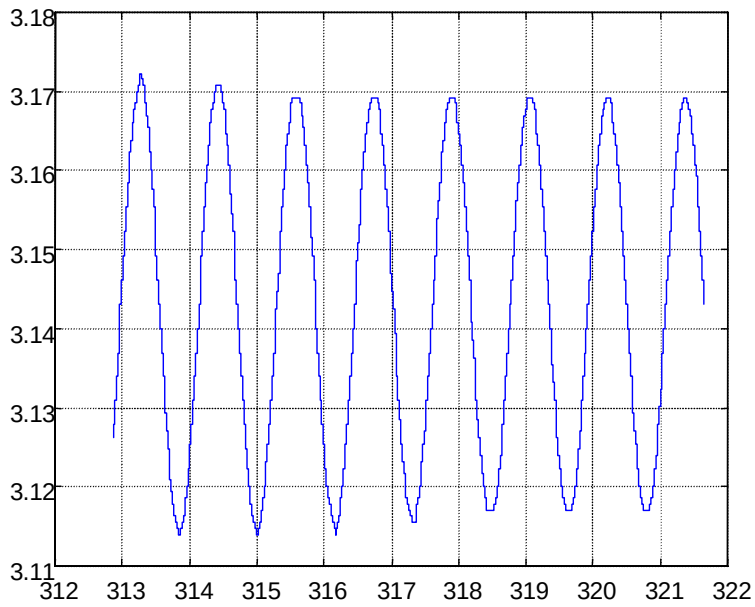
*fi(1, 1 : 4) = 0.25 * [1 1 1 1];*

pd_call('SetPendPosFilt', fi);

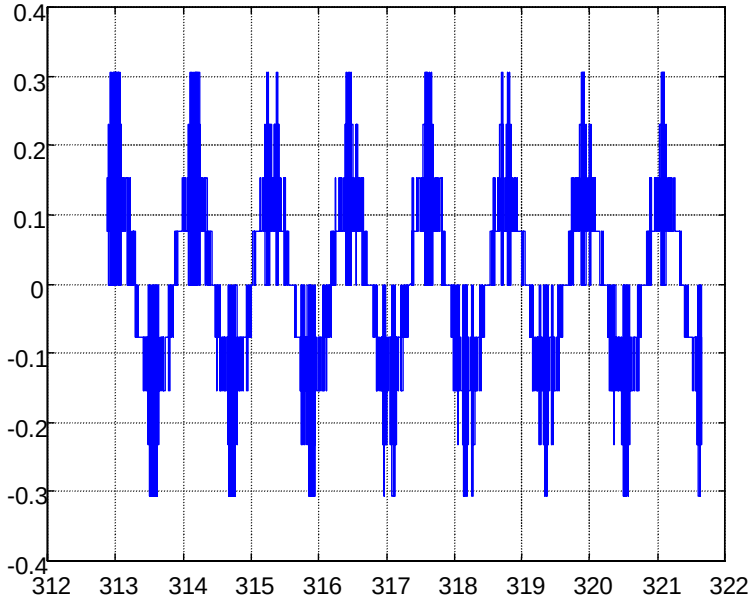
The following commands present the results of filtering:

h = pd_call('GetHistory');

stairs(h(1, :), h(2, :)); grid



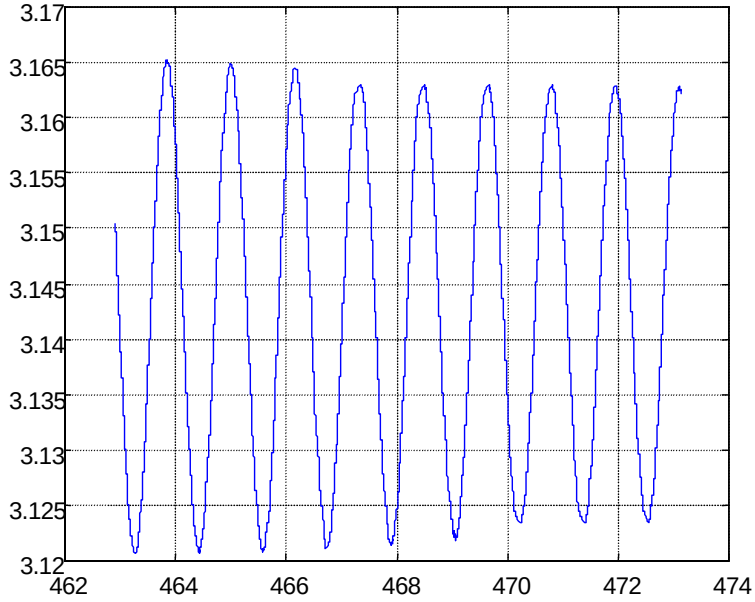
stairs(h(1, :), h(3, :)); grid



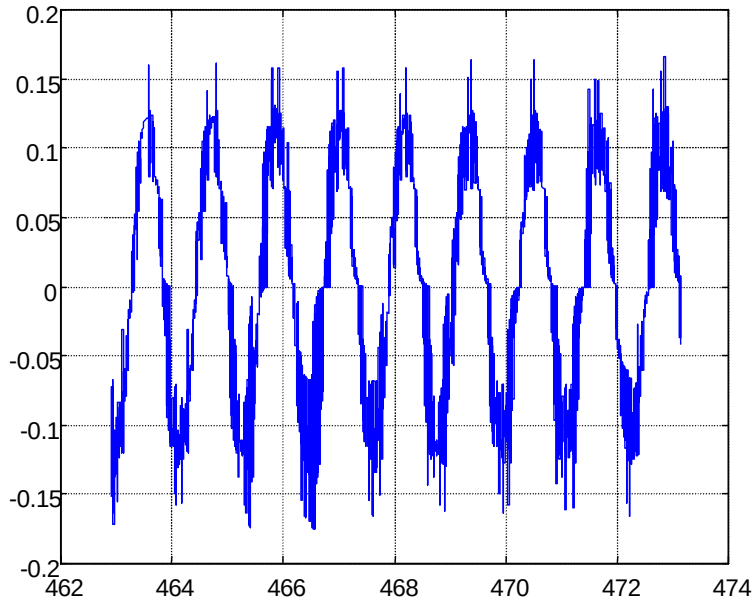
The velocity data looks better.

The following commands define and set a seven-order filter and present angle and angular velocity of the pole:

```
fi = pd_call( 'GetPendPosFilt' );
fi( 1, 1 : 8 ) = 0.125 * [1 1 1 1 1 1 1 1];
pd_call( 'SetPendPosFilt', fi );
pause( 10 )
h = pd_call( 'GetHistory' );
stairs( h( 1, : ), h( 2, : ) ); grid
```



stairs(h(1, :), h(3, :)); grid



Now the velocity looks acceptable.

2.25 SetPendSpeedFilt

Purpose: Set the parameters of the pole velocity digital filter.

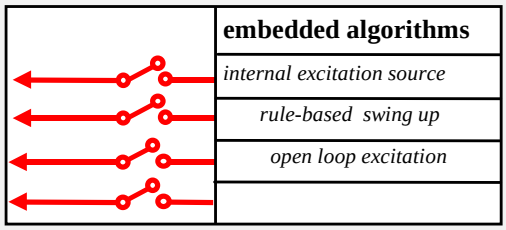
Synopsis: *fi = pd_call('SetPendSpeedFilt')*

Description: The function sets the parameters of the digital filter currently active in RTK. The filter associated with the *SetPendSpeedFilt* function operates on the data describing the velocity of the pole.

See description of the *GetCartPosFilt* for filter details.

See: *SetPendPosFilt, GetPendSpeedFilt*

2.26 SetPW



Purpose: Set parameters of internal excitation generator.

Synopsis: *pd_call('SetPW', Par)*

(The name *SetPW* means**:** set parameters of a wave.)

Description: The function sets the parameters of the internal excitation source. Next the number of the algorithm must be selected , and this starts the experiment using the pre-configured virtual excitation generator. The following periodical functions can be selected: constant, square, triangle, sinusoidal and random.

The elements of the *Par* argument have the following meaning:

Par(1) = 0: -constant value in this case (Fig. 2.7):

Par(2) - is the level in the range from -1.0 to +1.0,

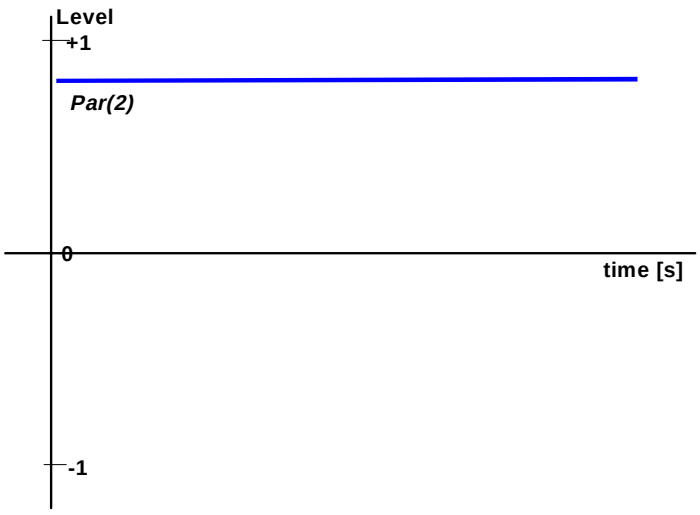


Fig. 2.7. Constant excitation.

Par(1) = 1: -for a square wave (Fig. 2.8):

Par(2), Par(3), Par(4), Par(5) - are the time periods

Par(6), Par(7), Par(8), Par(9) - are the appropriate levels

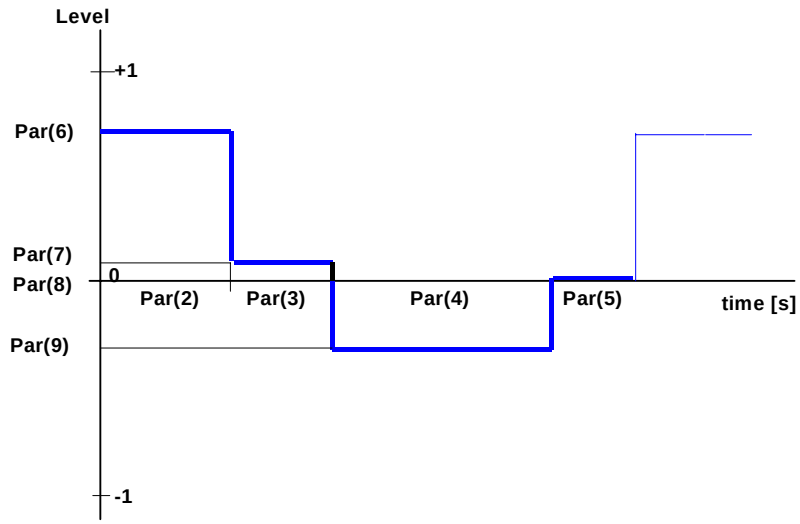


Fig. 2.8. Square excitation.

Par(1) = 2: - for a triangular wave (Fig. 2.9):

Par(2), Par(3) - are the time periods,

Par(4), Par(5) - are the appropriate levels,

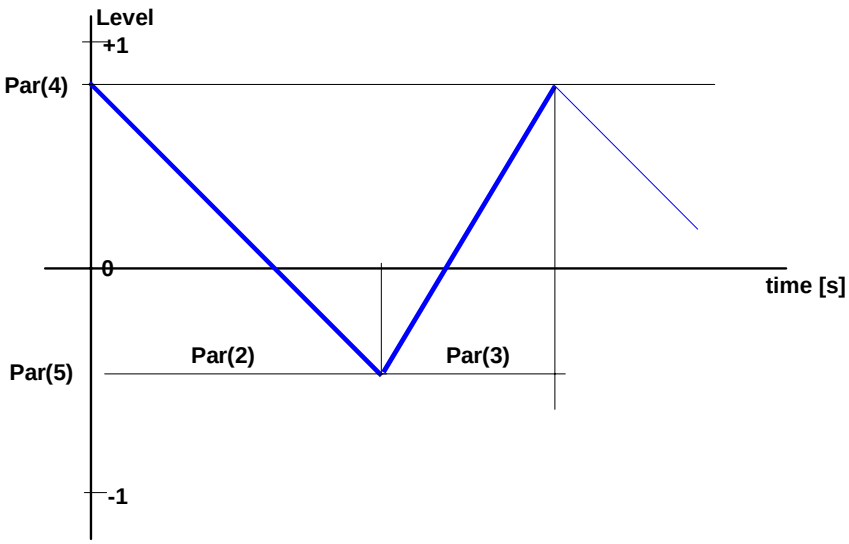


Fig. 2.9. Triangular excitation.

Par(1) = 3: -for a sinusoidal wave (Fig. 2.10):

Par(2) - is the time period,

Par(3), Par(4) -are the amplitudes for the upper and lower level,

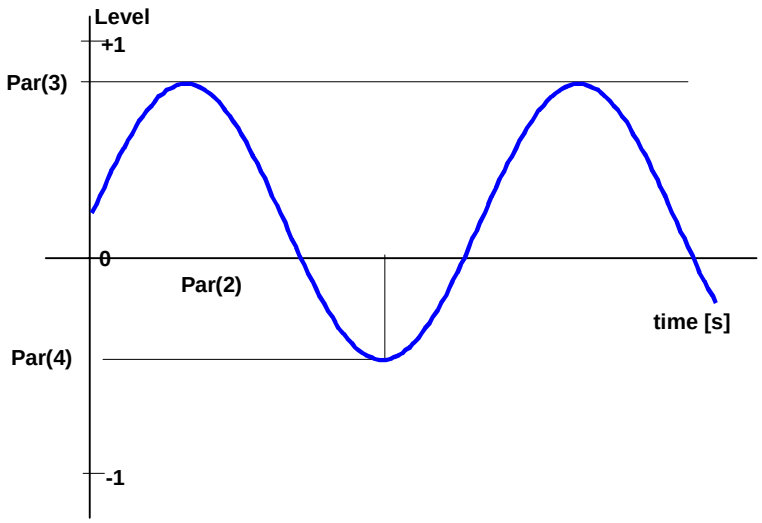


Fig. 2.10. Sinusoidal excitation.

Par(1) = 4: - for a random wave (Fig. 2.11):

Par(2) - is the time period when the output of the random generator is kept constant,

Par(3), Par(4) - are the minimum and maximum levels of the wave.

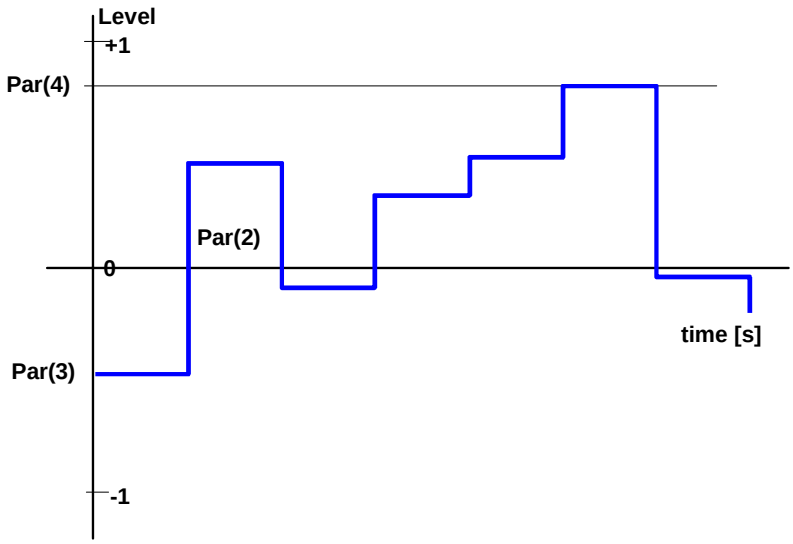
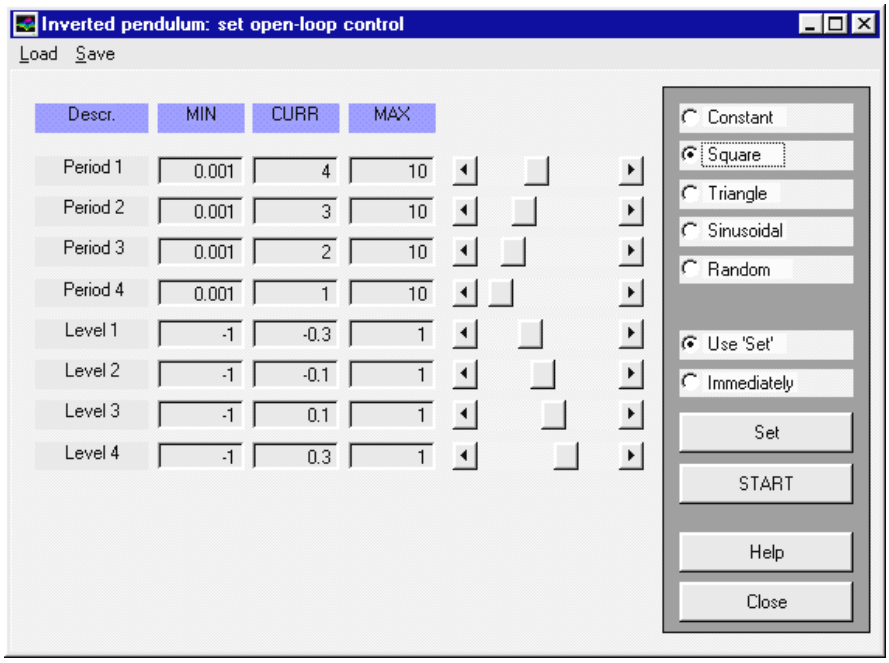


Fig. 2.11. Random excitation.

See: *GetPW, SetAlgNo*

MATLAB 5

Note: There is GUI tool which can be used to tune the parameters of the open-loop excitation. Execute the *pd_egen* command to call this tool (see figure below).



Example: Set the external excitation source to generate a sinusoidal wave: period equal to 4 seconds, and with a lower level of -0.3 and an upper level of 0.9.

```
Par = pd_call( 'GetPW' );
Par( 1 : 4 ) = [ 3  4  -0.3  0.9 ];
pd_call( 'SetPW', Par);
pd_call( 'GetPW' )
ans =
    3  4  -0.3  0.9  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
```

2.27 SetSampleTime

Purpose: Set basic clock.

Synopsis: *pd_call('SetSampleTime', Period)*

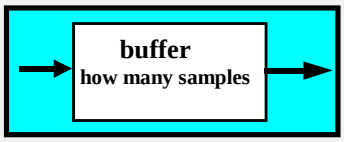
Description: The function sets the basic clock of RTK. The sampling period of A/D converter is set by the function. The controller output rate (D/A) can be equal to or greater than the basic clock frequency. The *Period* parameter must be in the range from 0.001s to 32.767s. The lower bound depends on the hardware configuration. The resolution is 0.001s.

See: *SetDivider, GetSampleTime*

Example: To set and to read the interval of the sampling period use the following statements:

```
pd_call( 'SetSampleTime', 0.025 );
pd_call( 'GetSampleTime' )
ans =
      0.025
pd_call( 'SetSampleTime', 0.0 );        % SampleTime<0.001 results
pd_call( 'GetSampleTime' )              % in 0.001
ans =
      0.001
```

2.28 StartAcq



Purpose: Clear content of the buffer.

Synopsis*:* *pd_call('StartAcq')*

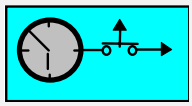
Description: The function erases the contents of the buffer. The action is similar to that of the *GetHistory* function, but does not return the content of the buffer. Notice, that the function *GetNoOfSamples* returns zero when invoked immediately after a *StartAcq* call**.**

See: *GetNoOfSamples*

Example*:* The call to the *GetNoOfSamples* function immediately after the call to the *StartAcq* function gives different results depending of sample period lengths and computer speed. For instance,

```
pd_call( 'SetSampleTime', 0.1 )
pd_call( 'StartAcq' ); pd_call( 'GetNoOfSamples' )
ans =
        0
pd_call( 'SetSampleTime', 0.01 )
pd_call( 'AtartAcq' ); pd_call( 'GetNoOfSamples' )
ans =
        3
```

2.29 StopPractical



Purpose: Stop practical.

Synopsis*:* *pd_call('StopPractical')*

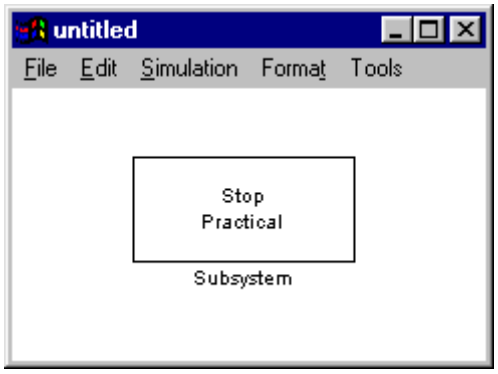
Description: The function is called to stop the practical. It suspends the currently active RTK control algorithm**.** The action is similar to

pd_call('SetAlgNo', 0).

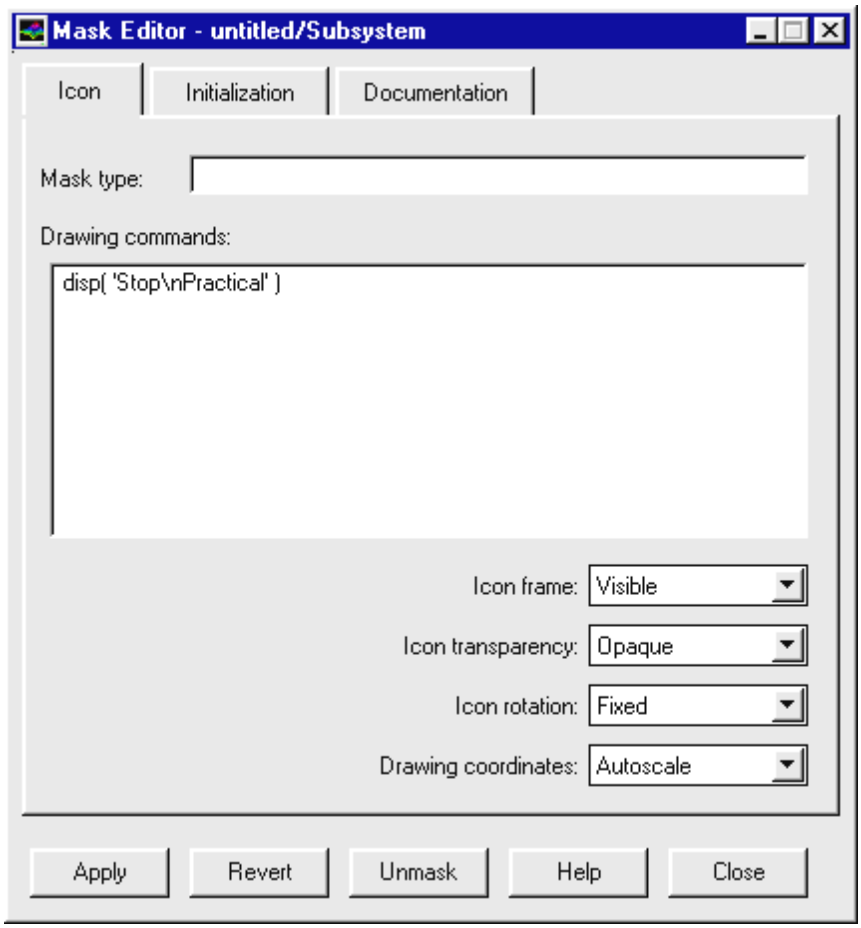
See: *SetAlgNo*

MATLAB 5

Example*:* You can create the Simulink model containing a single block. Double click over this block will cause execution of the *pd_call('StopPractical')* function. The Simulink model is presented below.



The *Stop Practical* block is masked. The mask of the *Subsystem* block sets the inscription inside the block and is shown below:

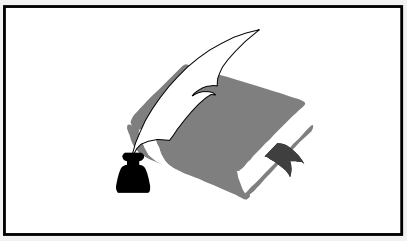


To join the *StopPractical* block with the *StopPractical* action the following command must be executed:

set_param('untitled/Subsystem', 'OpenFcn' , ... 'pd_call("StopPractical");')

This command sets the *OpenFcn* property of the *Subsystem* block from the *untitled* Simulink model to the *pd_call('StopPractical')* statement. It causes execution of the *pd_call('StopPractical')* command when the user double clicks over the *Subsystem* block.

2.30 UnloadLibrary



Purpose: Remove RTK library from memory.

Synopsis*:* *Count = pd_call('UnloadLibrary')*

Description: The function decrements the reference count of the loaded *pd_dll.dll* library module. When the reference count reaches zero, the library is removed from the memory. The reference counter value is returned.

MATLAB 5

In the MATLAB version 5 the call to the *UnloadLibrary* function removes immediately the RTK library from the memory.

See: *LoadLibrary*

MATLAB 4

Example: Remove pd_dll.dll library from memory (MATLAB ver. 4 only).

```
while ( pd_call( 'UnloadLibrary' ) > 0 )
    ;
end
```

3 Information Flow between Simulink Models and the Real-Time Kernel

This section describes an example of an S-function exchanging information between the Simulink model and the Real-Time Kernel. S-functions were developed to give Simulink the ability to construct generic simulation blocks, to handle different roles in one standard form, such as continuous simulation, discrete simulation, systems embedded within systems, and so on. In our applications special S-functions perform information exchange between Simulink models and the Real-Time Kernel.

This section explains the use of Simulink models as the graphical front-end of the real-time control program. It is important to realise that user-defined S-functions are at the heart of how your control experiments are performed. Inside a proper S-function you can select a real-time controller, set its parameters and make process data available within the MATLAB environment. The *masking* mechanism, available in Simulink, allows you to define a block in terms of its dialogue box, its icon and initialisation commands.

It is assumed that the reader is familiar with S-function formats but, if more details are required, refer to *"Simulink. Dynamic System Simulation Software. User's Guide"*.

MATLAB 5

The principles of data flow between Simulink models and the Real-Time Kernel differ in MATLAB version 4 and MATLAB version 5. There are different structures of S-functions as well.

3.1 SIMULINK MODELS FOR MATLAB VERSION 4

This section contains a description of Simulink models designed for MATLAB version 4. The Simulink models and S-functions are explained. Study this section only if you work with MATLAB ver. 4. For more information about MATLAB version 5 see section 3.2.

3.1.1 Guide for Simulink models and S-functions (examples)

We shall discuss the S-function of the *Swing, Upright and Downright Stabilisation* block. This block belongs to the main demo of the Simulink model called by the *pendulum* command in the MATLAB Command Window. The Simulink model executed after a double click on this block is presented in Fig. 3.1.

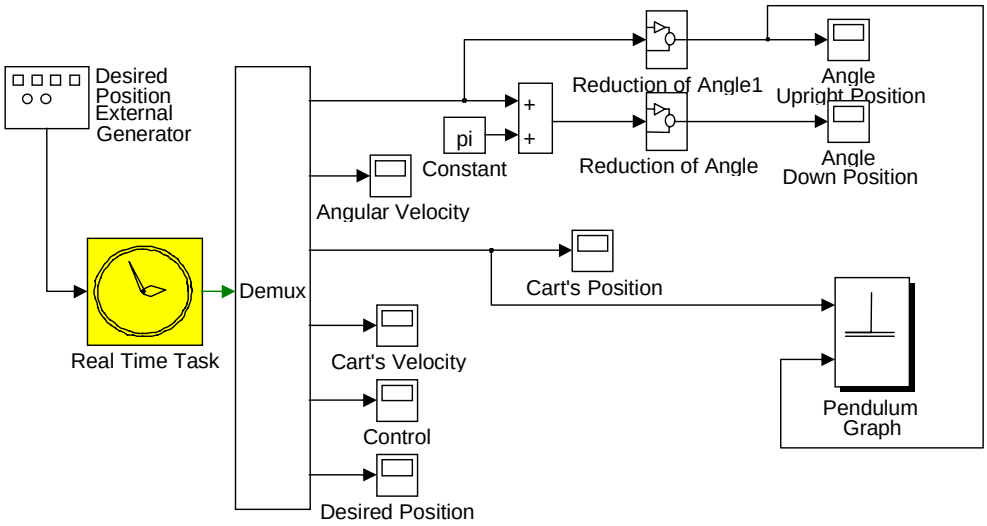


Fig. 3.1 Swing, Upright and Down Stabilisation block

Double click on the *Real-Time Task* block and the Real Time Kernel Dialogue box is displayed as shown below in Fig. 3.2.

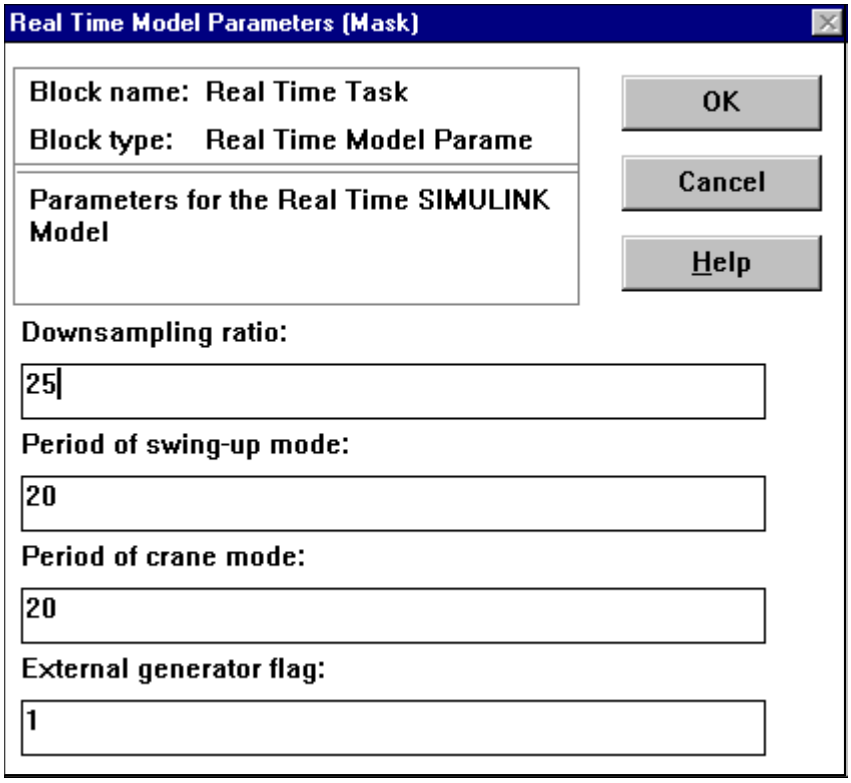


Fig. 3.2 Real-Time Kernel dialogue box.

The window shown in Fig. 3.2 sets the parameters of the Simulink model. The parameters are passed to the S-function during the execution of the model (after the *start* command from the Simulation menu).

The *Real-Time Kernel* block is masked. To see the details of this block one has to unmask it.

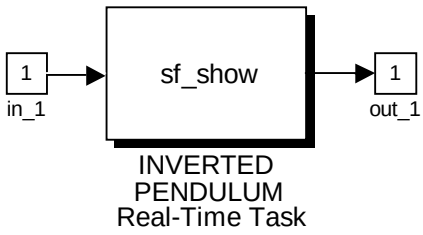


Fig. 3.3. Unmasked Real-Time Kernel block.

The *Real-Time Kernel* block contains one main block: S-function labelled as INVERTED PENDULUM Real-Time Task. The blocks *in_1* and *out_1* from Fig. 3.3 are required for

masking. The name of the S-function connected with the S-function block is *sf_show*. This S-function may be found in *sf_show.m* file.

A double click on the S-function block opens the following window (Fig. 3.4).

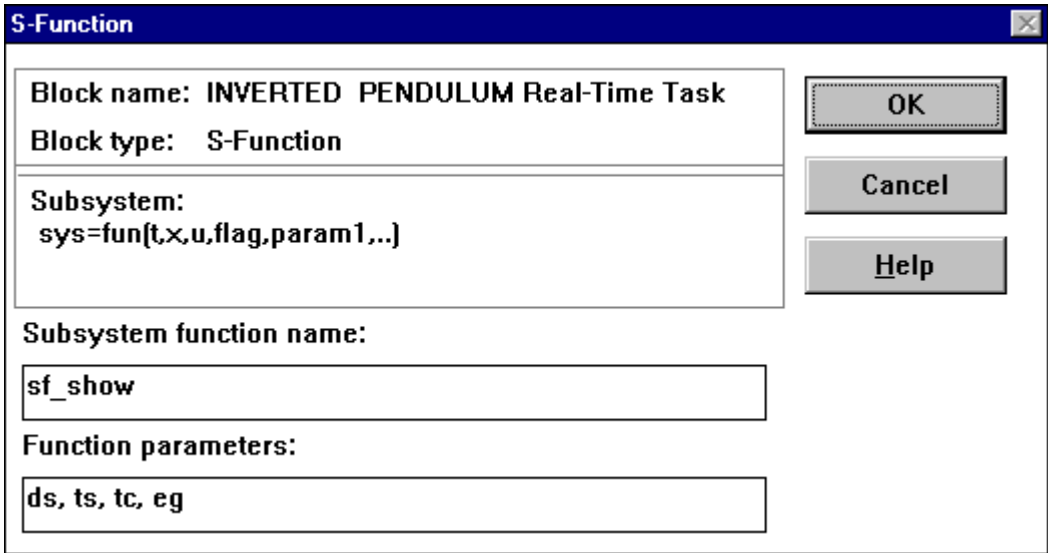


Fig. 3.4 S-function dialogue box.

In the window shown in Fig. 3.4 the user can set the subsystem S-function name and parameters. In this example the function name is *sf_show*. The names of the additional function parameters are set during masking of *Real-Time Kernel* block. See the *Initialisation commands* field in Fig. 3.5.

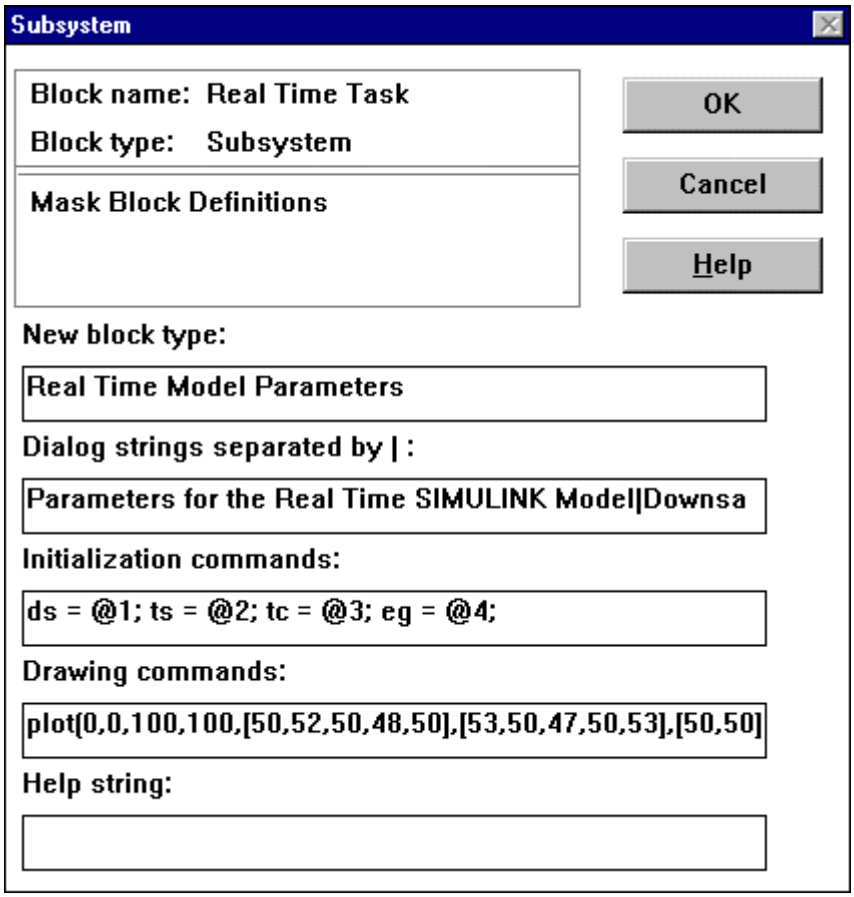


Fig. 3.5 Mask definition for *Real-Time Kernel* block.

3.1.2 S-function - example 1

The content of the *sf_show* S-function is described in the following part of this section. The lines written in italics are statements of the *sf_show.m* file. The lines marked on the left side contain comments added for explaining statements of the S-function.

The *sf_show* function changes control algorithms every *t_swing* and *t_crane* seconds. The demo starts from crane mode algorithm. After *t_crane* seconds algorithm is changed to swing-up and upright stabilisation mode. After *t_swing* seconds this S-function sets the algorithm back to the crane mode, and so on. If *eg_flag* differs from zero then *u* variable is used to set new value of the desired position of the cart. This function reads the history of the experiment and transfers samples to the output of the S-function block (1 point for every *downsamp* points).

function [sys, x0] = sf_show(t, x, u, flag, downsamp, t_swing, t_crane, eg_flag)

The function has following parameters:

t	- time,
x	- state vector,
u	- input to the S-function block,
flag	- the value passed to S-function by Simulink to distinguish different actions. The arguments *t*, *x*, *u* and *flag* are set and passed to S-function by Simulink automatically,
downsamp	- downsampling ratio. Defines how many samples are transferred to the output of the S-function block. For instance, if *downsamp* is equal to 10 only 1 sample of every 10 samples is transferred from Real-Time Kernel to the output of the S-function block,
t_swing	- period of swing-up and upright stabilisation mode. Defines time in seconds,
t_crane	- period of down stabilisation mode (crane). Determines time in seconds,
eg_flag	- external generator flag. If this value differs from zero then external generator sets desired position of the cart. The external generator is connected to the input of the S-function block.

```
global   pd_par   history   pos_in_history
```

➢ Global variables are used to store parameters of the controller, history of the experiment and auxiliary variables for downsampling.

```
if nargin==0 | flag==0    % Initialisation
sys=[ 0, 1, 6, 1, 0, 0 ];
```

➢ Set number of continuous states, number of discrete states, number of outputs and number of inputs (0 continuous states, 1 discrete state, 6 outputs, 1 input, 0 discontinuous roots, 0 direct feedthrough - without algebraic loops)

```
dummy = pd_call( 'SetAlgNo',      0 );
dummy = pd_call( 'SetP',          [ 0.10 -1.8 -1.0 0.0 0.0 0.35 0.5 ]);
dummy = pd_call( 'SetAlgNo',      3 );
```

➢ Set initial values of the control algorithm. First set the control of the DC motor to zero (control algorithm number 0). Then set parameters of the controller and activate control algorithm number 3.

dummy = pd_call('ResetTime', 0);

dummy = pd_call('SetDesPosition', 0);

➢ Reset experiment time and set desired position of the cart to the centre of the rail.

pd_par(1) = pd_call('GetSampleTime');

pd_par(2) = pd_call('GetDivider');

➢ Get sampling period and sampling time Divider

3.1.3 S-function - example 2

The second S-function executed by *Swing, Upright and Down Stabilisation* model is connected with *Pendulum Graph* block. The *Pendulum Graph* block is masked. Details of the unmasked block are given in Fig. 3.6.

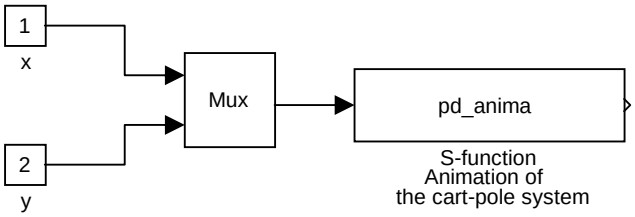


Fig. 3.6 Unmasked *Pendulum Graph* block.

It can be seen that this block executes the S-function included in *pd_anima.m* file.

When the *Pendulum Graph* block is masked the following window appears (Fig. 3.7).

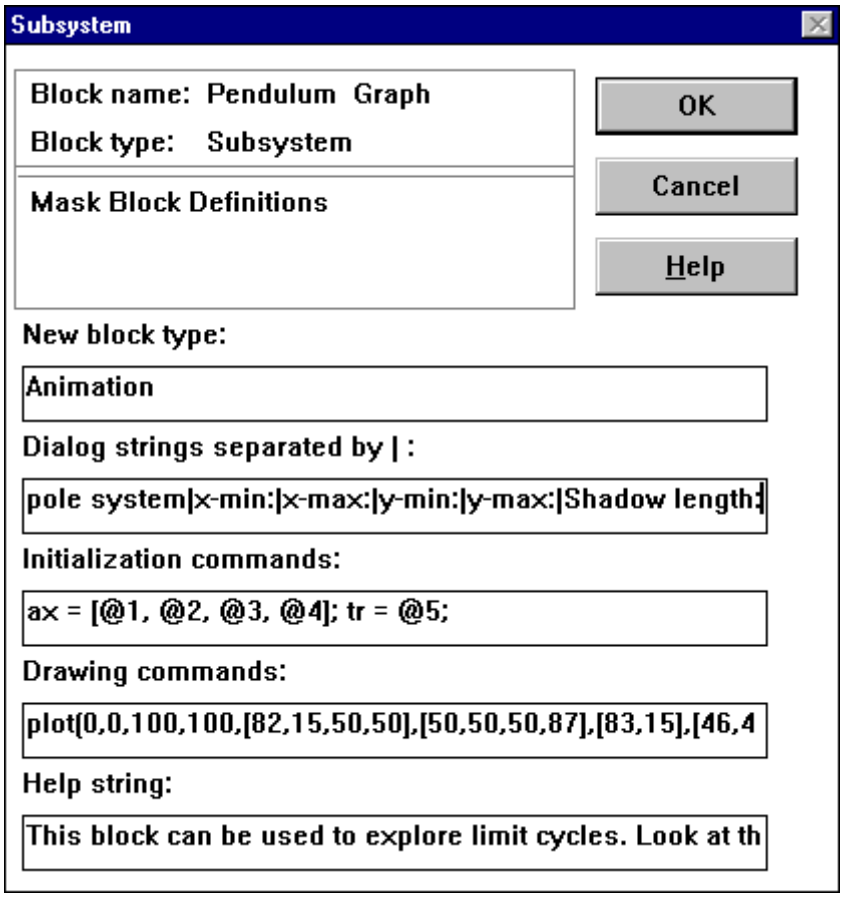


Fig. 3.7 Masking *Pendulum Graph* block.

Two variables are set: *ax* and *tr*. The first variable defines ranges of the X-axis and Y-axis of the animation window. The second contains number of shadow plots. When double clicking on the *Pendulum Graph* block the following window appears (Fig. 3.8)

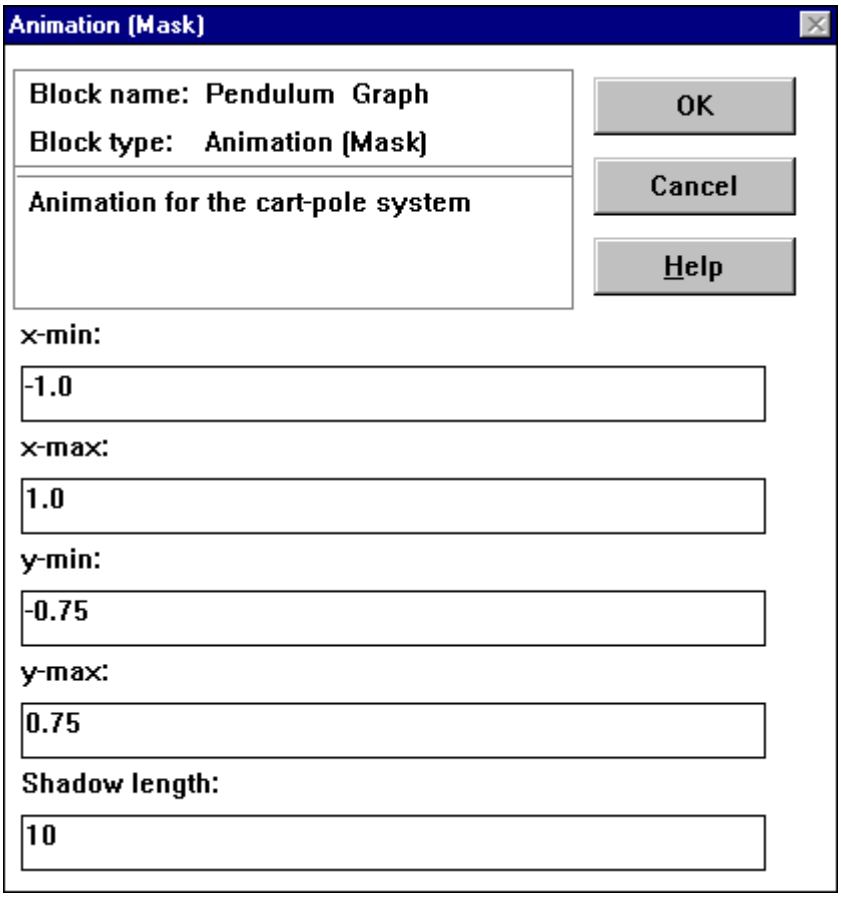


Fig. 3.8 Parameters of the *Pendulum Graph* block.

In this window ranges of axis and numbers of shadow plots can be set

The statements of *pd_anima* S-function are described below.

function [sys, x0] = sfunpend(t, x, u, flag, ax, tr)

This function creates the window and animates the position of the cart and the position of the pole. The shadows of the last *'tr'* positions of the pole are displayed.

The function has the following parameters:

t - time,

x - state vector,

u - input to the S-function block. It contains two values: position of the cart and angle of the pole,

flag - the value passed to S-function by Simulink to distinguish different actions. The arguments *t*, *x*, *u* and *flag* are set and passed to S-function by Simulink automatically,

ax - four-element vector determining the ranges of X-axis and Y-axis of the animation window,

tr - number of positions of the pole displayed as shadows.

```
fig_name = 'Animation of the cart-pole system';
shadow   = [ 0.5 0.5 0.5 ];
```

➢ Set the title of the animation window and colour of the shadow.

```
if ( flag  == 0 ) | (margin == 0),  % Initialisation
```

➢ Check, if the animation window has been created previously.

```
h_arr = get( 0, 'Children' );
   position = get(0, 'DefaultFigurePosition' );
   h = -99;
for i = 1 : length( h_arr )
     if strcmp(get( h_arr(i), 'Type'), 'figure')
       if strcmp( get( h_arr( i ), 'Name' ), fig_name )
         h = h_arr( i );
         delete( get(h_arr( i ), 'Children' ) );
```

```
            break;
          end;
        end;
      end;
```

➢ Find the handler of the animation window.

```
if ( h < 0 )          % Animation window does not exist
  h = figure( 'Name',  fig_name, 'NumberTitle', 'off',  'Color', 'w', 'Position',
position );
```

➢ Create a new window

```
 end;
axes( 'Xlim', ax(1:2), 'Ylim', ax(3:4),  'YColor', get( h, 'Color' ),  'XColor', 'k' );
```

➢ Set the ranges of the horizontal and vertical axis

```
line( 'EraseMode', 'none', 'Color', 'k', 'Xdata', [-1,  1], 'Ydata', [ 0.02, 0.02] );
line( 'EraseMode', 'none', 'Color', 'k', 'Xdata', [-1,  1], 'Ydata', [-0.02,-0.02] );
line( 'EraseMode', 'none', 'Color', 'k', 'Xdata', [-1, -1], 'Ydata', [-0.02, 0.02] );
line( 'EraseMode', 'none', 'Color', 'k', 'Xdata', [ 1,  1], 'Ydata', [-0.02, 0.02] );
```

➢ Plot the rail

```
for i = 1 : tr
    hd_arr( i ) = line( 'Visible', 'off', 'EraseMode', 'xor',  'LineWidth', 3 );
end;
```

➢ Prepare shadow lines

```
set( h, 'UserData', [ 0, hd_arr ] );
```

➢ Set *UserData* properties to *hd_add* variable

```
sys = [ 0; 0; 0; 2; 0; 0];  % System sizes - 2 inputs
```

➢ Initialize sizes

```
elseif flag == 2       % Calculate discrete state
   h_arr = get( 0, 'Children' );
```

```
    h = -99;
for i = 1 : length( h_arr )
    if strcmp(get( h_arr(i), 'Type'), 'figure')
      if strcmp( get( h_arr( i ), 'Name' ), fig_name )
        h = h_arr( i );
        usd = get( h, 'UserData' );
        pos = usd( 1 );
        hd_arr = usd( 2 : length( usd ) );
        if ( length( usd ) ~= tr + 1 )  % pos and hd_arr
          h = -999;
        end;
        break;
      end;
    end;
  end;
```

➢ Find the handler of the animation window.

```
if ( h > 0 )     % If the animation window exists
    if ( pos >= tr )
      set( hd_arr(tr), 'Color', shadow );
      hd_aux = hd_arr( 1 );
      hd_arr(1:tr-1) = hd_arr(2:tr);
      hd_arr( tr ) = hd_aux;
      set(hd_arr(tr),'xdata',[u(1),u(1)+0.5*sin(u(2))],  'ydata',[0,0.5*cos(u(2))], ...
            'EraseMode','xor', 'Color','r');
```

➢ Set previous position to shadow colour and destroy the last shadow.

```
  else
      pos = pos + 1;
      set(hd_arr(pos), 'xdata',[u(1),u(1)+0.5*sin(u(2))], 'ydata',[0,0.5*cos(u(2))],
            'Color','r', 'EraseMode','xor', 'Visible', 'on' );
      if ( pos > 1 )
```

```
        set(hd_arr(pos-1), 'Color', shadow );
      end;
    end
```

➢ Plot current position.

```
  set( h, 'UserData', [ pos, hd_arr ] );
end
  sys = [ ];
else
   sys = [ ];
end
```

3.2 SIMULINK MODELS FOR MATLAB VERSION 5

This section contains description of Simulink models designed for MATLAB version 5. The Simulink models and S-functions are explained. Study this section only if you work with MATLAB ver. 5.

3.2.1 Guide for Simulink models and S-functions (examples)

We shall discuss the S-function of the *Swing, Upright and Downright Stabilisation* block. This block belongs to the main demo of the *pendulum* Simulink model called by the *pendulum* command in the MATLAB Command Window. The Simulink model executed after a double click on this block is presented in Fig.3.9.

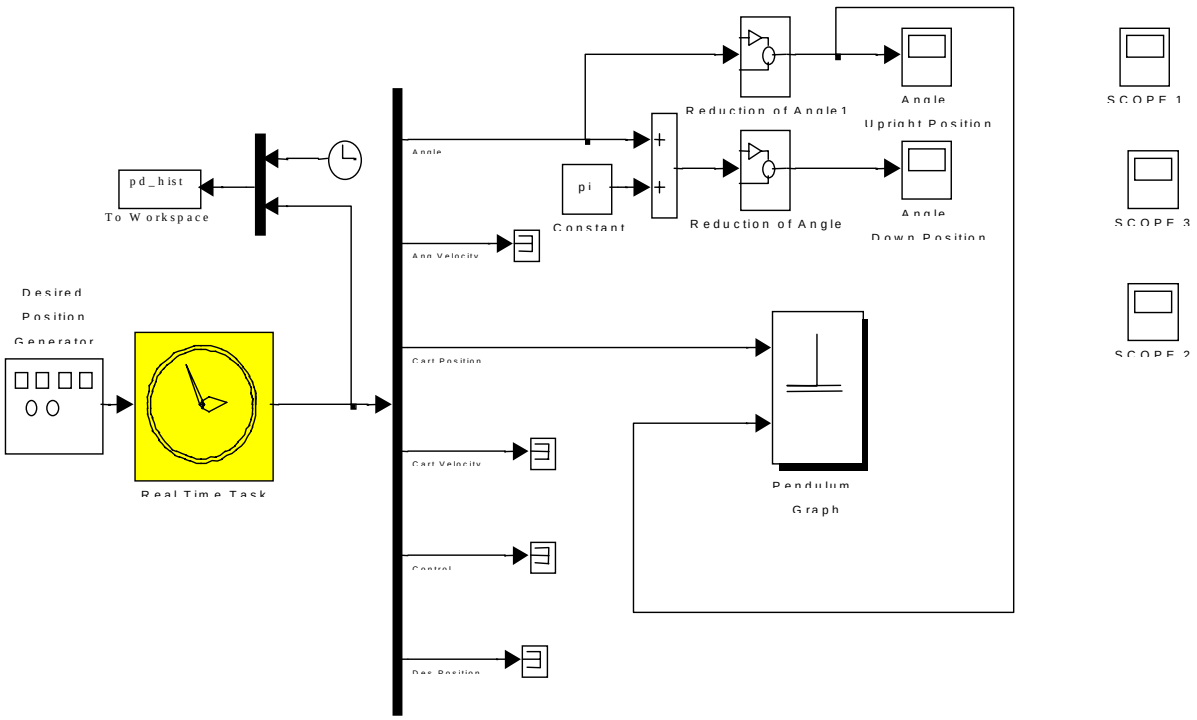


Fig. 3.9 *Swing, Upright and Down Stabilisation* block

Double click on the *Real-Time Kernel* block and you will see the following dialogue box (Fig.3.10).

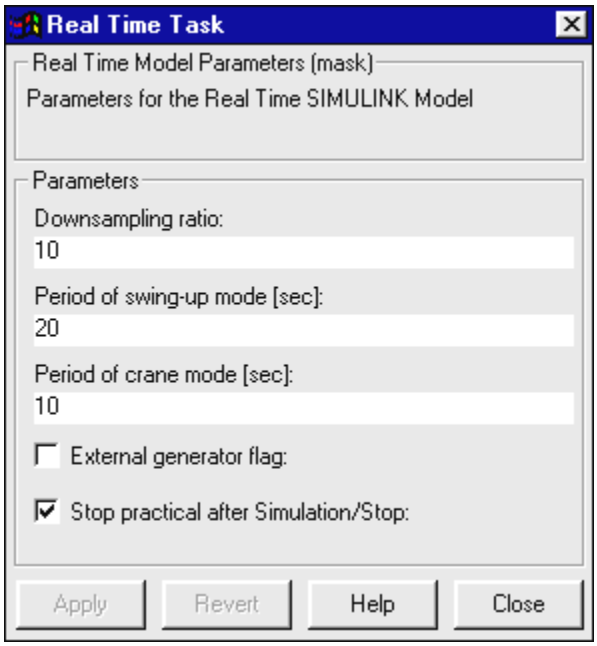


Fig. 3.10. Real-Time Kernel dialogue box.

The window shown in Fig. 3.10 sets parameters of the Simulink model. The parameters are passed to the S-function during the execution of the model (after the *start* command from the Simulation menu).

The *Real-Time Kernel* block is masked. To see the details of this block one has to execute the *Look under mask* command from *Edit* menu.

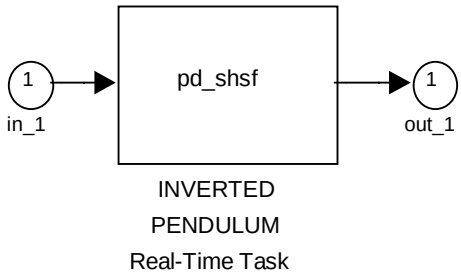


Fig. 3.11. Unmasked Real-Time Kernel block.

The *Real-Time Kernel* block contains one main block: S-function labelled as INVERTED PENDULUM Real-Time Task. The blocks *in_1* and *out_1* from Fig.3.11 are required for masking. The name of the S-function connected with the S-function block is *pd_shsf*. This S-function may be found in *pd_shsf.m* file.

A double click on the S-function block opens the following window (Fig.3.12).

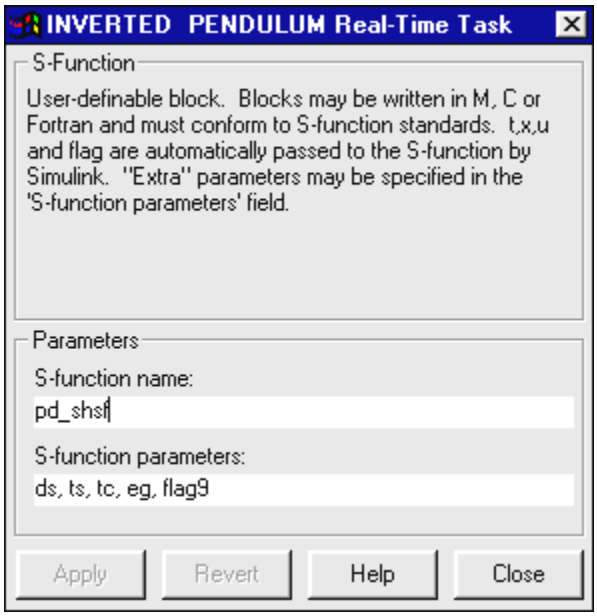


Fig. 3.12. S-function dialogue box.

The user can set the subsystem S-function name and parameters in the window shown in Fig.3.12. In this example the function name is *pd_shsf*. The names of the additional function parameters are set during masking of *Real-Time Kernel* block and are defined by *Edit Mask* command. See the *Initialisation* tab in Fig.3.13.

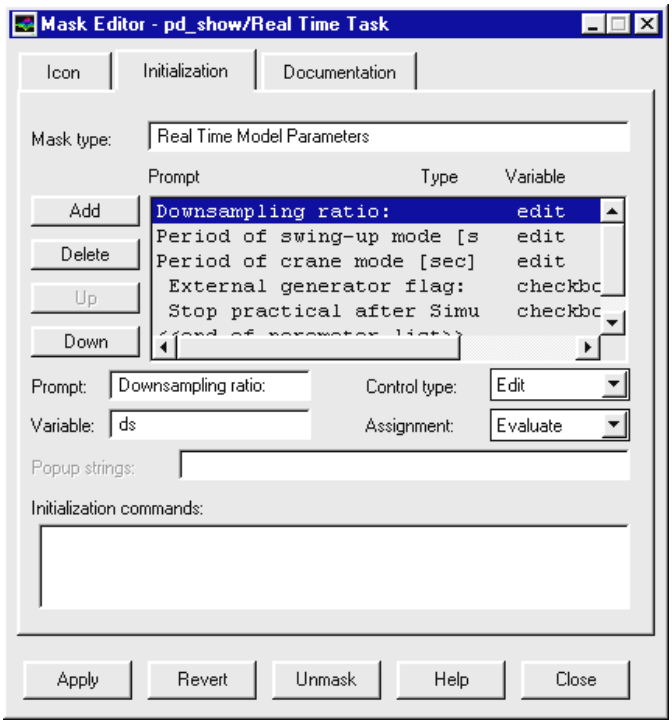


Fig. 3.13 Mask definition for *Real-Time Kernel* block.

3.2.2 S-function - example 1

The content of the *pd_shsf* S-function is described in the following part of this section. The lines written in italics are statements of the *pd_shsf.m* file. The lines marked on the left side contain comments added for explaining statements of the S-function.

The *pd_shsf* function changes control algorithms every *t_swing* and *t_crane* seconds. The demo starts from crane mode algorithm. After *t_crane* seconds algorithm is changed to swing-up and upright stabilisation mode. After *t_swing* seconds this S-function sets algorithm back to the crane mode, and so on. If *eg_flag* differs from zero then *u* variable is used to set new value of the desired position of the cart. The *pd_shsf* function reads history of the experiment and transfers samples to the output of S-function block (1 sample for every *downsamp* samples). The *flag9* variable is used to switch-off control after *Simulation/Stop* command.

function [sys,x0,str,ts] = sfunc(t, x, u, flag, ds, t_swing, t_crane, eg_flag, flag9)

The function has following parameters:

t - time,

x - state vector,

u - input to the S-function block,

flag - the value passed to S-function by Simulink to distinguish different actions. The arguments *t*, *x*, *u* and *flag* are set and passed to S-function by Simulink automatically,

ds - downsampling ratio. Defines how many samples are transferred to the output of the S-function block. For instance, if *downsamp* is equal to 10 only 1 sample of every 10 samples is transferred from Real-Time Kernel to the output of the S-function block,

t_swing - period of swing-up and upright stabilisation mode. Defines time in seconds,

t_crane - period of down stabilisation mode (crane). Determines time in seconds,

eg_flag - external generator flag. If this value differs from zero then external generator sets desired position of the cart. The external generator is connected to the input of the S-function block.

flag9 - flag used to switch-off simulation. If *flag9* is set to 1 the control is off after executing *Simulation/Stop* command

```
global history pd_flag9
```

➢ Global variables are used to store parameters of the controller, history of the experiment and auxiliary variable for downsampling.

```
switch flag,
case 0,    % Initialisation
        % call simsizes for a sizes structure, fill it in and convert it to a sizes array
sizes = simsizes;
        % Set number of continuous states, number of discrete states, number of outputs and
        % number of inputs.   0 continuous states, 1 discrete state, 6 outputs, 1 inputs
sizes.NumContStates  = 0;
sizes.NumDiscStates  = 1;
sizes.NumOutputs     = 6;
sizes.NumInputs      = 1;
sizes.DirFeedthrough = 0;
sizes.NumSampleTimes = 1;     % at least one sample time is needed
sys = simsizes(sizes);
```

➢ Set number of continuous states, number of discrete states, number of outputs and number of inputs (0 continuous states, 1 discrete state, 6 outputs, 1 input, 0 discontinuous roots, 0 direct feedthrough - without algebraic loops, 1 sample time)

```
pd_flag9 = flag9;
```

➢ Store *flag9* variable for future usage

```
% Set initial values of the control algorithm
dummy = pd_call('SetAlgNo', 0 );
p = pd_call( 'GetP' );
p( 1 : 7 ) = [ 0.10 -1.8 -1.0 0.0 0.0 0.35 0.5 ];
p = pd_call( 'SetP', p );
dummy = pd_call('SetAlgNo', 3 );   % LQ
```

➢ Set initial values of the control algorithm. First set control of the DC motor to zero (control algorithm number 0). Then set parameters of the controller and activate control algorithm number 3.

```
dummy = pd_call('ResetTime' );
dummy = pd_call('SetDesPosition', 0 );
```

➢ Reset experiment time and set desired position of the cart to the centre of the rail.

```
dummy = pd_call('SetSampleTime', 0.01);
```

➢ Set sample period

```
while ( pd_call( 'GetNoOfSamples' ) <= ds )
    ;
end;
history=pd_call( 'GetHistory' )';
```

➢ Wait for the first sample which may be sent to the output

```
x0 = max( history(:,1) );
```

➢ Set initial conditions

```
str = [];          % str is always an empty matrix
                   % initialise the array of sample times
ts  = [-2 0];      % variable sample time

case 2,                    % Update - set new reference position
```

➢ Check the value of discrete state flag. This S-function doesn't need to use discrete state. When the *flag* equals to 2 the S-function changes the active algorithm and sets desired position of the cart.

```
if( rem( t, t_swing + t_crane ) < t_crane )
```

➢ Detect which algorithm is active in the current time point.

```
dummy = pd_call( 'SetAlgNo', 0 );
p = pd_call( 'GetP' );
p( 1 : 7 ) = [ 0.10 -1.8 -1.0 0.0 0.0 0.35 0.5 ];
dummy = pd_call( 'SetP',     p );
dummy = pd_call( 'SetAlgNo', 3 );          % LQ
```

```
else
    dummy = pd_call( 'SetAlgNo', 0 );
p = pd_call( 'GetP' );
    p( 1 : 10 ) = [ 0.10 0.8 8.0 1.4 0.6 0.35 0.33 8.0 0.4 0.3 ];
    dummy = pd_call( 'SetP',     p );
    dummy = pd_call( 'SetAlgNo', 2 );     % rule-based
end;
```

➢ Set control to zero (select control algorithm number 0), set parameters of the new control algorithm and finally, activate new algorithm.

```
if( ne( eg_flag, 0 ) )
dummy = pd_call('SetDesPosition', u );
end
```

➢ Set desired position of the cart when *eg_flag* is not equal to zero. The desired value is passed to the function as *u* variable.

```
sys = x;
```

➢ Return dummy value. This action is required by the S-function.

```
case 3,    % Outputs - return samples
```

➢ Calculate output from the S-function block. The downsample coefficient *ds* is used to select the sample which is returned as the output from the S-function block. The *ds* variable is used to send only one sample of every *ds* samples from Real-Time Kernel to the output.

```
 sys = history( ds, 2:7 );
curr_len = size( history, 1 );
history = history( ds + 1 : curr_len, : );


case 4,    % GetTimeOfNextVarHit
```

➢ Calculate next discrete time point. The S-function block will be activated in this time point.

➢ Wait for the next sample which can be transferred *curr_len = size(history, 1);*

```
if( curr_len < ds )
```

```
while ( pd_call( 'GetNoOfSamples' ) + curr_len <= ds )
                ;
    end;
    history = [ history; pd_call( 'GetHistory' )' ];
end
```

➢ from the Real-Time Kernel to the output of the S-function.

```
sys = history( ds, 1 );
```

➢ The next discrete time point is taken from real-time data acquisition buffer.

The This action synchronises the simulation time of the Simulink model and the real time of the Real-Time Kernel.

```
case 9,                         % Terminate
```

➢ Terminate simulation. The *pd_flag9* variable is used to switch off the control value. This variable is set to 1 if the *Stop practical after Simulation/Stop* check box is chosen (see Fig. 3.10)

```
if( eq( pd_flag9, 1 ) )         % Checked
dummy = pd_call( 'SetAlgNo', 0 );
dummy = pd_call( 'SetSampleTime', 0.05 );
end
sys = [];


case 1,                         % Unhandled flags
sys = [];
```

➢ The flag equal to 1 is not used.

```
otherwise                       % Unexpected flags
error(['Unexpected flag = ',num2str(flag)]);
end
```

3.2.3 S-function - example 2

The second S-function executed by *Swing, Upright and Downright Stabilisation* model is connected with the *Pendulum Graph* block. The block belongs to the main demo of the *pendulum* Simulink model. The *Pendulum Graph* block is masked. Details of the unmasked block are given in Fig.3.14.

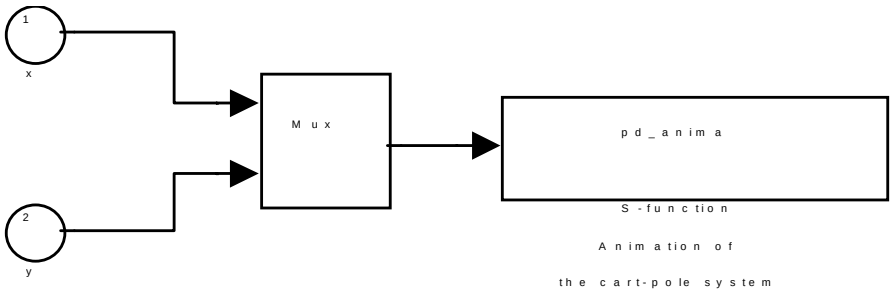


Fig. 3.14. Unmasked *Pendulum Graph* block.

It can be seen that this block executes the S-function included in *pd_anima.m* file.

When you mask the *Pendulum Graph* block the following window appears (Fig. 3.15)

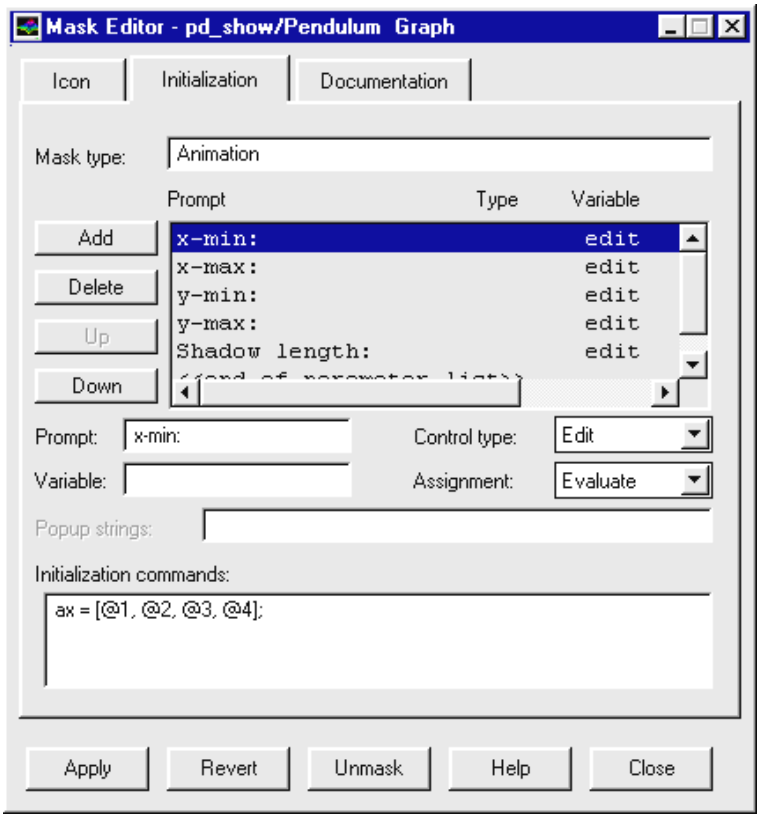


Fig. 3.15 Masking *Pendulum Graph* block.

Two variables are set: *ax* and *tr*. The first variable defines ranges of the X-axis and Y-axis of the animation window. This is visible on Fig.3.15. The second contains the number of shadow plots. When double clicking on the *Pendulum Graph* block the following window appears (Fig.3.16)

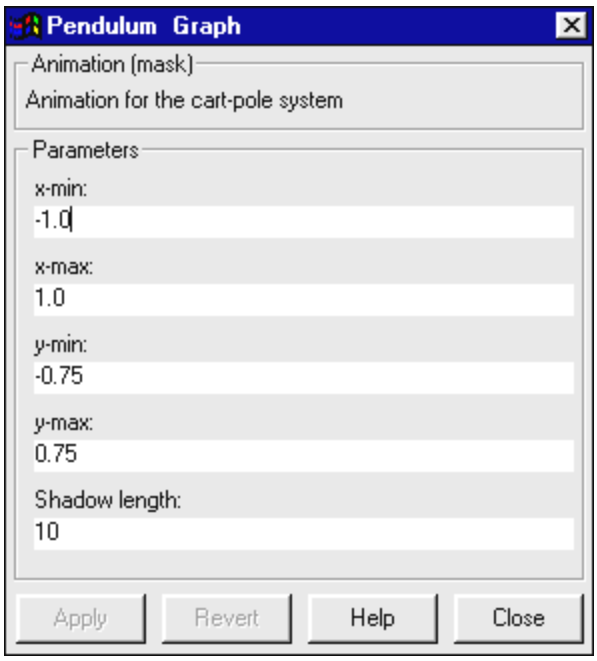


Fig. 3.16 Parameters of the *Pendulum Graph* block.

In this window the ranges of axis and number of shadow plots can be set

The statements of *pd_anima* S-function are described below.

function [sys,x0,str,ts] = sfunc(t, x, u, flag, ax, tr)

This function creates the window and animates the position of the cart and position of the pole. The shadows of last *'tr'* positions of the pole are displayed.

The function has the following parameters:

t - time,

x - state vector,

u - input to the S-function block. It contains two values: position of the cart and angle of the pole,

flag - the value passed to the S-function by Simulink to distinguish different actions. The arguments *t*, *x*, *u* and *flag* are set and passed to S-function by Simulink automatically,

ax - four-element vector determining the ranges of X-axis and Y-axis of the animation window,

tr -number of positions of the pole displayed as shadows.

fig_name = 'Animation of the cart-pole system';

shadow = [0.5 0.5 0.5];

➢ Set title of the animation window and colour of the shadow.

```
switch flag,
case 0,    % Initialisation
```

➢ Initialise sizes - 0 continuous states, 0 discrete state, 0 outputs, 2 inputs

```
sizes = simsizes;
sizes.NumContStates  = 0;
sizes.NumDiscStates  = 0;
sizes.NumOutputs     = 0;
sizes.NumInputs      = 2;
sizes.DirFeedthrough = 0;
sizes.NumSampleTimes = 0;     % at least one sample time is needed

sys = simsizes(sizes);

x0  = [ ];
str = [ ];    % str is always an empty matrix
ts  = [ ];    % [-2 0];     variable sample time
```

➢ Check, if the animation window has been created previously.

```
h_arr = get( 0, 'Children' );
position=get(0, 'DefaultFigurePosition' );
h = -99;
```

```
for i = 1 : length( h_arr )
if strcmp(get( h_arr(i), 'Type'), 'figure')
        if strcmp( get( h_arr( i ), 'Name' ), fig_name )
                h = h_arr( i );
                delete( get(h_arr( i ), 'Children' ) );
                break;
        end;
end;
end;
```

➢ Find the handler of the animation window.

```
if ( h < 0 )
```

➢ Create a new window

```
h = figure( 'Name',  fig_name, 'NumberTitle', 'off', ...
        'Color', 'w',      'Position',    position );
end;
```

➢ Set the ranges of the horizontal and vertical axis

```
axes( 'Xlim', ax(1:2), 'Ylim', ax(3:4),  'YColor', get( h, 'Color' ), 'XColor', 'k' );
```

➢ Plot the rail

```
line( 'EraseMode', 'none', 'Color', 'k', 'Xdata', [-1,  1], 'Ydata', [ 0.02, 0.02] );
line( 'EraseMode', 'none', 'Color', 'k', 'Xdata', [-1,  1], 'Ydata', [-0.02,-0.02] );
line( 'EraseMode', 'none', 'Color', 'k', 'Xdata', [-1, -1], 'Ydata', [-0.02, 0.02] );
line( 'EraseMode', 'none', 'Color', 'k', 'Xdata', [ 1,  1], 'Ydata', [-0.02, 0.02] );
```

➢ Prepare shadow lines

```
for i = 1 : tr
hd_arr( i ) = line( 'Visible', 'off', 'EraseMode', 'xor', 'LineWidth', 3 );
end;
```

➢ Set *UserData* properties to *hd_add* variable

```
set( h, 'UserData', [ 0, hd_arr ] );
```

```
case 2,    % Update state
```

➢ Find the handler of the animation window.

```
h_arr = get( 0, 'Children' );
h = -99;
for i = 1 : length( h_arr )
if strcmp(get( h_arr(i), 'Type'), 'figure')
        if strcmp( get( h_arr( i ), 'Name' ), fig_name )
                h = h_arr( i );
                usd = get( h, 'UserData' );
                pos = usd( 1 );
                hd_arr = usd( 2 : length( usd ) );
                if ( length( usd ) ~= tr + 1 )  % pos and hd_arr
                        h = -999;
                end;
                break;
        end;
end;
end;
if ( h > 0 )                    % If animation window exists
if ( pos >= tr )
```

➢ Set previous position to shadow colour and destroy the last shadow.

```
        set( hd_arr(tr), 'Color', shadow );
        hd_aux = hd_arr( 1 );
        hd_arr(1:tr-1) = hd_arr(2:tr);
        hd_arr( tr ) = hd_aux;
        set(hd_arr(tr),'xdata',[u(1),u(1)+0.5*sin(u(2))], 'ydata',[0,0.5*cos(u(2))], ...
                'EraseMode','xor', 'Color','r');
else
```

➢ Plot current position.

```
        pos = pos + 1;
```

```
        set(hd_arr(pos), 'xdata',[u(1),u(1)+0.5*sin(u(2))],...
                'ydata',[0,0.5*cos(u(2))], 'Color','r', ...
                'EraseMode','xor',  'Visible', 'on' );
        if ( pos > 1 )
                set(hd_arr(pos-1), 'Color', shadow );
        end;
end
set( h, 'UserData', [ pos, hd_arr ] );
end
sys = [ ];

case { 1, 3, 4, 9 },                % Unhandled flags
sys = [ ];
otherwise                           % Unexpected flags
error(['Unexpected flag = ',num2str(flag)]);
end
```

Notes

4 Guide to the Graphical User Interface (Examples)

The graphical user interface (GUI) is a set of user-configurable graphical objects , such as menus, buttons, lists and fields. MATLAB provides functions that enable a user to add features of a GUI to the MATLAB application. This section describes these features and shows how to use them for tuning or changing the form of the application.

MATLAB GUI elements fall into two classes:

- Controls
- Menus

In this manual only the Control class of GUI elements will be presented using example applications. Controls are graphics objects such that, when manipulated with the mouse cause a desired action. A control can perform an action with immediate results or can change settings that modify future actions.

MATLAB 5

The way of creating GUI elements illustrated in this section is mainly used in the MATLAB version 4 environment. In MATLAB version 5 the *guide* tool should be used.

4.1 TYPES OF CONTROLS

- **Push Buttons** are small screen objects usually labelled with text. Clicking the mouse button on a push button causes MATLAB to perform a defined action.
- **Check Boxes** let the user select one or more options among available alternatives. Check boxes act like toggle switches, indicating the state of “on” or “off”. The state is “on” if the box is checked.
- **Radio Buttons** let the user choose among mutually exclusive alternatives. Like check boxes, radio buttons act as toggle switches, indicating the state of either “on” or “off”.
- **Sliders** let the user choose a value within a range of values. Sliders are devices which display their values graphically. The user can change the value by moving the indicator.
- **Pop-up Menus** let the user choose an item from a list. Pop-up menus are used for choosing a value from a list of values.
- **Static Text** displays a single line of textual information. It is typically used to label a group of other controls, provide direction for the user, or indicate values associated with a slider.
- **Editable Text** let the user enter a string value to be used by the application. The value can consist of one or more lines of text. The user can accept, edit, delete, or replace an editable text value.
- **Frames** let the user enclose regions of the figure window in a frame. You can then place other controls within the frame to group them visibly.

To create controls use the *uicontrol* function:

h=uicontrol(hfig, ‘PropertyName’,’PropertyValue’,...)

hfig - specifies the figure handler, where a control is defined. If you omit the figure handler, the control appears in the current figure window. If there is no figure window, MATLAB creates a new one.

PropertyName/PropertyValue are pairs describing controls. When specifying properties, you need to enter only enough characters to uniquely identify the property. Examples of the properties are given below.

4.2 HOW TO CREATE AND USE CONTROLS (EXAMPLES)

In the first example we create a figure window (Fig. 4.1), where controls used by *Pendulum Interface* are defined (see: *Parameters of the Controller* block in the main control window). At the beginning, a figure handler is created:

```
figpend=figure('Name','PENDULUM INTERFACE',...
        'Units','Normalized',...
    'Pos',[ 0.25 0.6 0.5 0.15],'Color',[ 0.1 0.8 0.2],'Num','off','Menu','none',...
        'Resize','on','NextPlot','new');
```

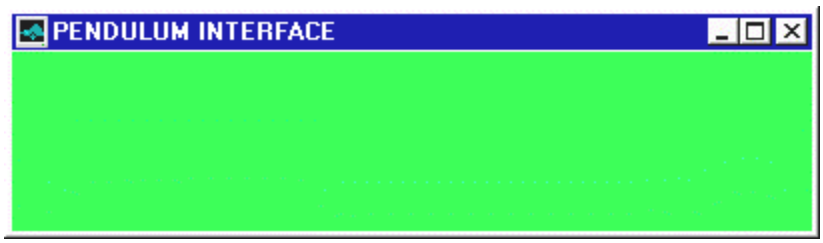


Fig. 4.1 Window definition in GUI

figpend - is the name of the figure handler, which is used in the next example.

Name, Units, Pos, Num, Menu, Resize, NextPlot are property names. The strings to be defined after this are property values.

The function *get(h,'PropertyName')* returns the value of *PropertyName* for the graphic object with the handle *h*.

Function *set(h,'PropertyName','PropertyValue')* sets the value of *PropertyName* as *PropertyValue* for the graphic object with the handle *h*.

➢ You can use this function in your MATLAB applications, e.g.:

get(figpend,'Resize') - get value of *Resize* property. This value is equal to *'on'*.

set(figpend,'Pos',[0.1 0.2 0.7 0.6]); - change position of the figure window.

➢ Now, we can create a GUI control of the slider class (Fig. 4.2):

```
slid=uicontrol(figpend,...              % handler of figure window
        'Style', 'Slider',...           % type of control - Slider
        'Units','Normalized',...
```

```
'Position',[ 0.05 0.5 0.3 0.2],...   % define position and size
'Min',-1,...                         % define the minimum of range
'Max',1,...                          % define the maximum of range
'Value',0,...                        % current value of a slider
'CallBack',['‘,...
    'p=pd_call(lib,"getp");’...
    'p(1)=get(slid,"Value")’;...
    'a=pd_call(lib,"setp",p);']);
```

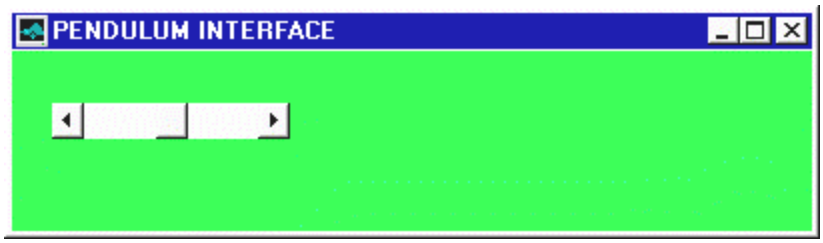


Fig. 4.2 Window and slider definition in GUI

The property *CallBack* defines the action that MATLAB performs when the user activates the slider. *CallBack* must be a string.

The property *CallBack* in this example loads parameters from RTK, then changes the value of *friction (p(1)),* and finally sends new parameters to RTK.

The following part of the code creates an editable text which realises the same tasks as the slider control described above (Fig. 4.3).

```
editp=uicontrol(figpend,...
    'Style', 'Edit',...                  % type of control - Edit
    ‘Units’,’Normalized’,...
    'Position',[ 0.45 0.5 0.15 0.2],...
    'String',num2str(0),...             % current value of editp
    'CallBack',['‘,...
        'p=pd_call(lib,"getp");’...
        'p(1)=str2num(get(editp,’’String"))’;...
```

```
'a=pd_call(lib,"setp",p);']);
```

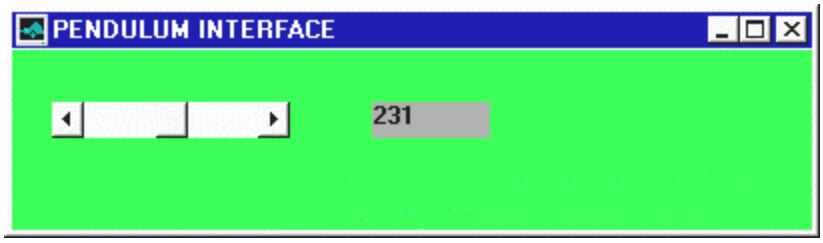


Fig. 4.3 Window, slider and text definition in GUI

Notice, that the value entered by the user must be converted to a number first.

The user can set the initial value of the slider or editable text by the current value of parameter described creating these controls and modifying *Value* or *String* properties:

```
p=pd_call(lib,'getp');        % vector p containing the current values of parameters
'Value',p(1),...              % modification of slider :
'String',num2str(p(1)),...    % modification of editable text
```

The following code creates pop-up menus which choose a pendulum control algorithm from a given list (Fig. 4.4).

```
contr=uicontrol(figpend,...
        'Style','Popup',...
        'Units','Normalized',...
    'String','None|Open Loop|Swing up|Crane',...
    'Position',[ 0.65 0.5 0.3 0.2],...
    'CallBack',[' aux = get(contr,"Value");',...
                'if (aux == 1) ',...
                ...
                'elseif (aux == 2) ',...
            ...
                'elseif (aux == 3)',...
                ...
                'elseif (aux == 4)',...
```

...

'end']);

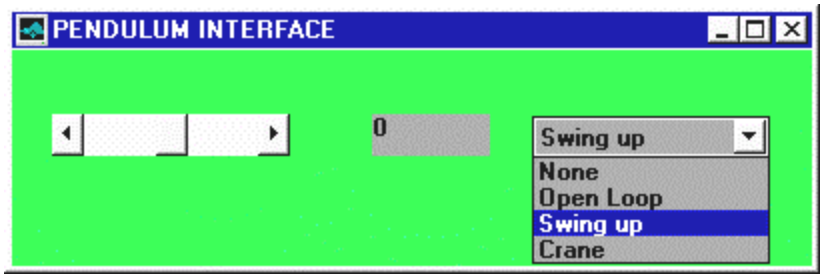


Fig. 4.4 Window, slider, text and control algorithm definition in the GUI

You can specify a list in a pop-up menu using a *String* property by separating individual items with a vertical bar (|). The *Value* property contains the index of the item selected from the pop-up menu list. For example, if the user selects the item *Crane* from the list then *Value* takes the index 4.

5 Quick Reference Table

Function Name	Description
1. RTK communication	
GetAlgNo	number of algorithm
GetBaseAddress	get base address of PCL-812 board
GetCartPosFilt	get parameters of the cart position filter
GetCartSpeedFilt	get parameters of the cart velocity filter
GetDesPosition	desired position
GetDivider	auxiliary clock multiplier
GetHistory	content of the buffer
GetNrOfSamples	number of samples in the buffer
GetP	parameters of the control algorithm
GetPendPosFilt	get parameters of the pole position filter
GetPendSpeedFilt	get parameters of the pole velocity filter
GetPW	parameters of internal excitation generator
GetSampleTime	basic clock period
LoadLibrary	load RTK DLL library
ResetEncoder	reset incremental encoders
ResetTime	set experiment's time to zero
SetAlgNo	selects the control algorithm and starts the experiment
SetBaseAddress	set base address of PCL-812 board
SetCartPosFilt	set parameters of the cart position filter
SetCartSpeedFilt	set parameters of the cart velocity filter
SetDesPosition	set desired position from the keyboard
SetDivider	set auxiliary clock
SetP	set parameters of the controller

SetPendPosFilt	set parameters of the pole position filter
SetPendSpeedFilt	set parameters of the pole velocity filter
SetPW	set parameters of internal excitation generator
SetSampleTime	set basic clock
StartAcq	clear content of the buffer
StopPractical	stop practical
UnloadLibrary	remove RTK DLL library from memory

2. Demo (m-files and Simulink models)	
pendulum	start main demo
pd_cpar	select active controller; set parameters of active controller (GUI interface)
pd_dpar	set desired position of the cart (GUI interface)
pd_egen	set parameters of open-loop excitation (GUI interface)
pd_setad	set base address (GUI interface)
pd_stet	set sample period; plot measurements (GUI interface)
pd_lqcr	LQ control algorithm in crane mode (Simulink model)
pd_pid	PID control algorithm in crane mode (Simulink model)
pd_show	swing-up and crane mode control algorithm (Simulink model)
pd_up	swing-up and upright stabilisation control algorithm (Simulink model)